Unit 4 Data visualization in data science

Introduction to data visualization,

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data.

What is Data Visualization?

Data visualization translates complex data sets into visual formats that are easier for the human brain to comprehend. This can include a variety of visual tools such as:

- **Charts**: Bar charts, line charts, pie charts, etc.
- **Graphs**: Scatter plots, histograms, etc.
- **Maps**: Geographic maps, heat maps, etc.
- **Dashboards**: Interactive platforms that combine multiple visualizations.

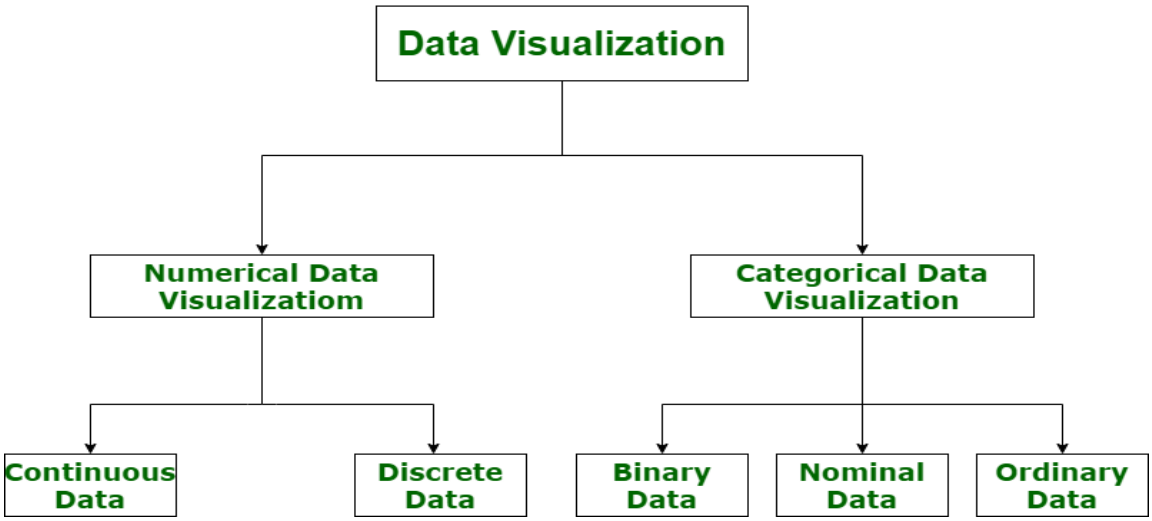
The primary goal of data visualization is to make data more accessible and easier to interpret, allowing users to identify patterns, trends, and outliers quickly. This is particularly important in the context of big data, where the sheer volume of information can be overwhelming without effective visualization techniques.

Types of Data for Visualization

Performing accurate visualization of data is very critical to market research where both numerical and categorical data can be visualized, which helps increase the impact of insights and also helps in reducing the risk of analysis paralysis. So, data visualization is categorized into the following categories:

- Numerical Data
- Categorical Data

Let’s understand the visualization of data via a diagram with its all categories.



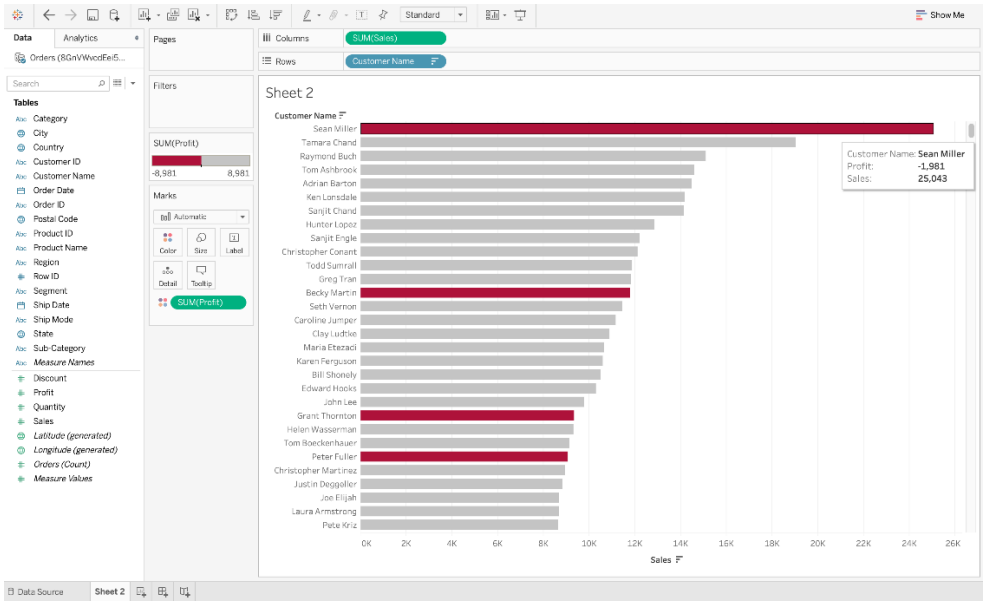
Why is Data Visualization Important?

Let's take an example. Suppose you compile visualization data of the company's profits from *2013 to 2023* and create a line chart. It would be very easy to see the line going constantly up with a drop in just 2018. So you can observe in a second that the company has had continuous profits in all the years except a loss in 2018.

It would not be that easy to get this information so fast from a data table. This is just one demonstration of the usefulness of data visualization. Let's see some more reasons why visualization of data is so important.

1. Data Visualization Discovers the Trends in Data

The most important thing that data visualization does is discover the trends in data. After all, it is much easier to observe data trends when all the data is laid out in front of you in a visual form as compared to data in a table. For example, the screenshot below on visualization on Tableau demonstrates the sum of sales made by each customer in descending order. However, the color red denotes loss while grey denotes profits. So it is very easy to observe from this visualization that even though some customers may have huge sales, they are still at a loss. This would be very difficult to observe from a table.



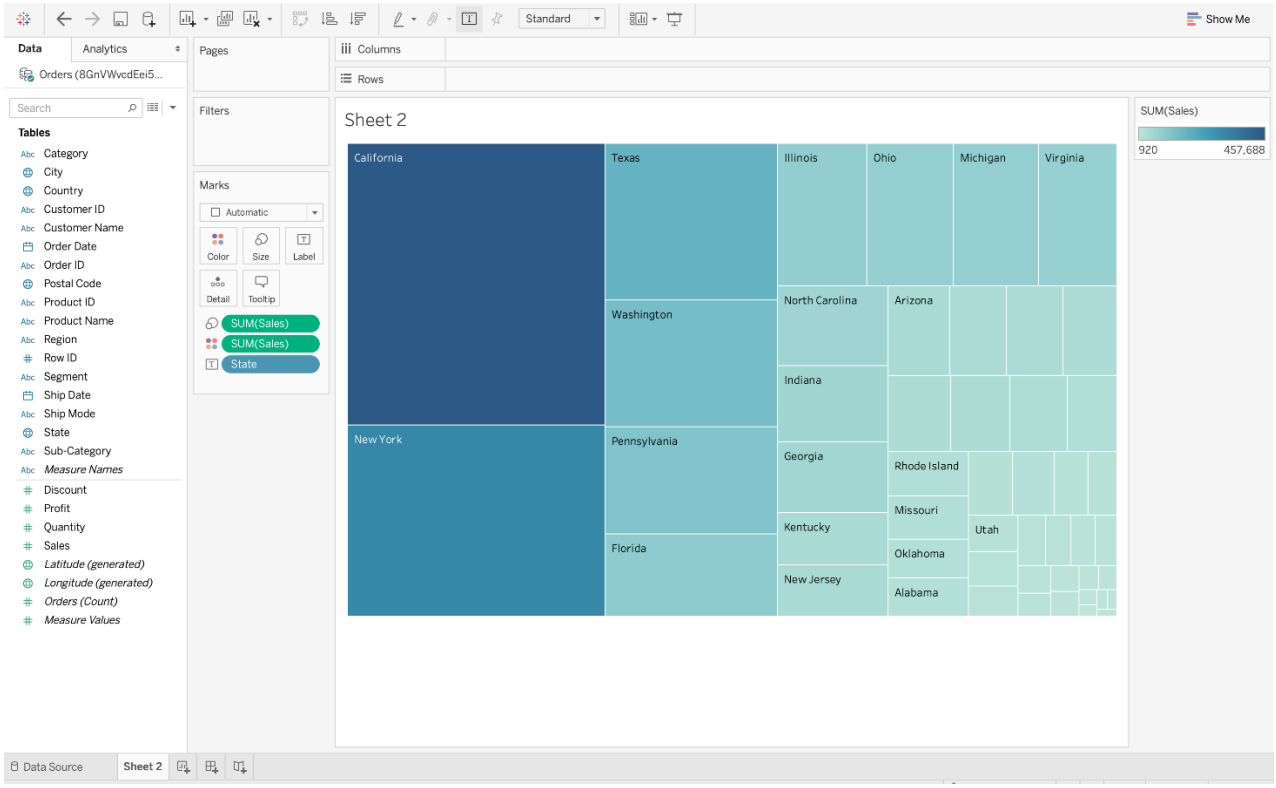
2. Data Visualization Provides a Perspective on the Data

Visualizing Data provides a perspective on data by showing its meaning in the larger scheme of things. It demonstrates how particular data references stand concerning the overall data picture. In the data visualization below, the data between sales and profit provides a data perspective concerning these two measures. It also demonstrates that there are very few sales above 12K and higher sales do not necessarily mean a higher profit.



3. Data Visualization Puts the Data into the Correct Context

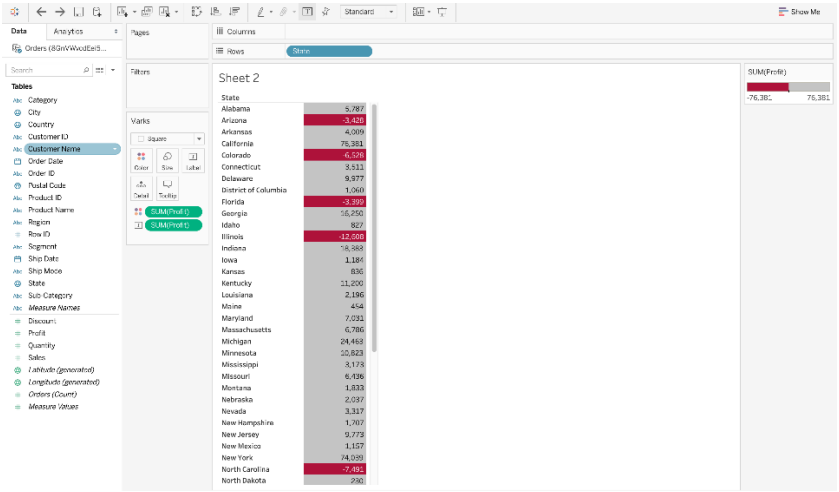
It isn't easy to understand the context of the data with data visualization. Since context provides the whole circumstances of the data, it is very difficult to grasp by just reading numbers in a table. In the below data **visualization on Tableau**, a TreeMap is used to demonstrate the number of sales in each region of the United States. It is very easy to understand from this data visualization that California has the largest number of sales out of the total number since the rectangle for California is the largest. But this information is not easy to understand outside of context without visualizing data.



4. Data Visualization Saves Time

It is definitely faster to gather some insights from the data using data visualization rather than just studying a chart. In the screenshot below on Tableau, it is very easy to identify the states

that have suffered a net loss rather than a profit. This is because all the cells with a loss are coloured red using a heat map, so it is obvious states have suffered a loss. Compare this to a normal table where you would need to check each cell to see if it has a negative value to determine a loss. Visualizing Data can save a lot of time in this situation!



5. Data Visualization Tells a Data Story

Data visualization is also a medium to tell a data story to the viewers. The visualization can be used to present the data facts in an easy-to-understand form while telling a story and leading the viewers to an inevitable conclusion. This data story, like any other type of story, should have a good beginning, a basic plot, and an ending that it is leading towards. For example, if a data analyst has to craft a data visualization for company executives detailing the profits of various products, then the data story can start with the profits and losses of multiple products and move on to recommendations on how to tackle the losses.

Now, that we have understood the basics of Data Visualization, along with its importance, now will be discussing the Advantages, Disadvantages and Data Science Pipeline (along with the diagram) which will help you to understand how data is compiled through various checkpoints.

Types of Data Visualization Techniques

Various types of visualizations cater to diverse data sets and analytical goals.

1. **Bar Charts:** Ideal for comparing categorical data or displaying frequencies, bar charts offer a clear visual representation of values.
2. **Line Charts:** Perfect for illustrating trends over time, line charts connect data points to reveal patterns and fluctuations.
3. **Pie Charts:** Efficient for displaying parts of a whole, pie charts offer a simple way to understand proportions and percentages.
4. **Scatter Plots:** Showcase relationships between two variables, identifying patterns and outliers through scattered data points.
5. **Histograms:** Depict the distribution of a continuous variable, providing insights into the underlying data patterns.

6. **Heatmaps:** Visualize complex data sets through color-coding, emphasizing variations and correlations in a matrix.
7. **Box Plots:** Unveil statistical summaries such as median, quartiles, and outliers, aiding in data distribution analysis.
8. **Area Charts:** Similar to line charts but with the area under the line filled, these charts accentuate cumulative data patterns.
9. **Bubble Charts:** Enhance scatter plots by introducing a third dimension through varying bubble sizes, revealing additional insights.
10. **Treemaps:** Efficiently represent hierarchical data structures, breaking down categories into nested rectangles.
11. **Violin Plots**: Violin plots combine aspects of box plots and kernel density plots, providing a detailed representation of the distribution of data.
12. **Word Clouds**: Word clouds are visual representations of text data where words are sized based on their frequency.
13. **3D Surface Plots**: 3D surface plots visualize three-dimensional data, illustrating how a response variable changes in relation to two predictor variables.
14. **Network Graphs**: Network graphs represent relationships between entities using nodes and edges. They are useful for visualizing connections in complex systems, such as social networks, transportation networks, or organizational structures.
15. **Sankey Diagrams**: Sankey diagrams visualize flow and quantity relationships between multiple entities. Often used in process engineering or energy flow analysis.

Visualization of data not only simplifies complex information but also enhances decision-making processes. Choosing the right type of visualization helps to unveil hidden patterns and trends within the data, making informed and impactful conclusions.

Tools for Visualization of Data

The following are the 10 best Data Visualization Tools

1. Tableau
2. Looker
3. Zoho Analytics
4. Sisense
5. IBM Cognos Analytics
6. Qlik Sense
7. Domo
8. Microsoft Power BI

9. Klipfolio
10. SAP Analytics Cloud

Advantages and Disadvantages of Data Visualization

Advantages of Data Visualization:

- **Enhanced Comparison:** Visualizing performances of two elements or scenarios streamlines analysis, saving time compared to traditional data examination.
- **Improved Methodology:** Representing data graphically offers a superior understanding of situations, exemplified by tools like Google Trends illustrating industry trends in graphical forms.
- **Efficient Data Sharing:** Visual data presentation facilitates effective communication, making information more digestible and engaging compared to sharing raw data.
- **Sales Analysis:** Data visualization aids sales professionals in comprehending product sales trends, identifying influencing factors through tools like heat maps, and understanding customer types, geography impacts, and repeat customer behaviors.
- **Identifying Event Relations:** Discovering correlations between events helps businesses understand external factors affecting their performance, such as online sales surges during festive seasons.
- **Exploring Opportunities and Trends:** Data visualization empowers business leaders to uncover patterns and opportunities within vast datasets, enabling a deeper understanding of customer behaviors and insights into emerging business trends.

Disadvantages of Data Visualization:

- **Can be time-consuming:** Creating visualizations can be a time-consuming process, especially when dealing with large and complex datasets.
- **Can be misleading:** While data visualization can help identify patterns and relationships in data, it can also be misleading if not done correctly. Visualizations can create the impression of patterns or trends that may not exist, leading to incorrect conclusions and poor decision-making.
- **Can be difficult to interpret:** Some types of visualizations, such as those that involve 3D or interactive elements, can be difficult to interpret and understand.
- **May not be suitable for all types of data:** Certain types of data, such as text or audio data, may not lend themselves well to visualization. In these cases, alternative methods of analysis may be more appropriate.
- **May not be accessible to all users:** Some users may have visual impairments or other disabilities that make it difficult or impossible for them to interpret visualizations. In these cases, alternative methods of presenting data may be necessary to ensure accessibility.

Role of Visualization in Data Analysis,

As AI and machine learning become increasingly ubiquitous, data visualization is more necessary than ever. With its help, teams can comprehend the performance of their models as well as pinpoint areas that need optimization. It is becoming a vital component of any successful data analytics strategy, as businesses need to leverage this powerful tool to stay competitive in an ever-evolving marketplace.

Data visualization plays a critical role in data analysis by transforming raw data into meaningful, easy-to-understand visual formats. It is an essential tool for analysts, decision-makers, and stakeholders to extract insights and make data-driven decisions. Here's how visualization impacts the data analysis process:

1. Simplifying Complex Data

- Raw data, especially when it's vast and complex, can be overwhelming and hard to interpret. Visualization distills this data into graphs, charts, and diagrams, making it easier to spot trends, correlations, and patterns. For example, a large dataset of monthly sales can be visualized as a line graph, clearly showing trends over time, which would be hard to grasp from a table of numbers.

2. Identifying Patterns and Trends

- Visualization helps analysts quickly identify key patterns, trends, and anomalies in the data. For example, a line chart can help to see how sales or website traffic changes over time, showing seasonality, growth trends, or dips. Similarly, a scatter plot might reveal correlations between two variables, such as age and income, in an easy-to-understand format.

3. Supporting Decision-Making

- Visualized data aids decision-makers in making informed decisions based on the analysis. When data is presented visually, insights become more accessible and actionable, leading to quicker, more effective decisions. For instance, dashboards showing key performance indicators (KPIs) can give managers a real-time view of business health, helping them adjust strategies as needed.

4. Communicating Insights

- Effective communication of data insights is crucial, especially when the audience is non-technical. Data visualizations make it easier to convey complex findings in an engaging way. Instead of presenting raw data or statistics, you can use charts, graphs, or infographics to tell a clear story. This is especially important when presenting to stakeholders who may not have a deep understanding of data analysis.

5. Facilitating Exploratory Data Analysis (EDA)

- Before performing complex statistical tests, analysts often engage in exploratory data analysis (EDA) to understand the dataset. Visualization is a vital part of this process.

Tools like histograms, scatter plots, and box plots allow analysts to explore distributions, outliers, and potential relationships between variables, helping them decide on the next steps for deeper analysis.

6. Revealing Relationships Between Variables

- Visualizations can help uncover relationships or correlations between different variables. For example, a scatter plot can highlight the linear or non-linear relationship between two variables, such as the relationship between advertising spend and sales. A heatmap might show how different variables are correlated, helping identify the most important predictors.

7. Making Data Accessible

- Data visualization makes large and complex datasets more accessible. For example, in big data environments, it's hard to analyze every data point manually. However, visualizations such as bar charts, heatmaps, and geospatial maps can summarize data in a form that's easier to interpret and understand. This is particularly helpful in fields like healthcare, finance, or marketing, where large datasets are the norm.

8. Improving Data Quality and Cleaning

- Visualization can help identify data quality issues like missing values, duplicates, or outliers. For example, box plots can highlight outliers in a dataset, and bar charts might reveal imbalances in categorical data. By visualizing the data, analysts can catch these issues early and take corrective actions before proceeding with deeper analysis or modeling.

9. Enhancing Comparisons

- Visualizations allow for straightforward comparisons across categories or variables. For instance, bar charts or stacked bar charts are often used to compare values across different groups, such as comparing sales performance across regions. This comparison can easily highlight which categories or periods performed better or worse.

10. Facilitating Predictive Analysis

- Visualization can also play a key role in predicting future outcomes by visualizing trends and forecasting. A time series graph, for instance, can show historical data that allows for trend analysis and future predictions. By recognizing these trends visually, analysts can provide data-driven forecasts and inform strategic planning.

11. Engaging the Audience

- Data visualizations, especially interactive ones (like dashboards or web-based charts), keep the audience engaged by allowing them to explore the data themselves. Interactive elements enable users to drill down into specific data points, which can lead to more personalized insights.

12. Improving Data Interpretation

- Visualizations make data interpretation more intuitive. Human brains are wired to process visual information more efficiently than raw numbers or text, so well-designed charts or diagrams often lead to faster and better understanding. They also help avoid common pitfalls in interpreting numerical data, such as misreading or overlooking subtle patterns.

Principles of Effective Data Visualization,

1. Clarity

The visualization should be clear and easily understood by the intended audience.

2. Simplicity

Keep the visualization simple and avoid unnecessary complexity.

3. Purposeful

Understand what message or insight you want to communicate and design for that purpose.

4. Consistency

Maintain consistency in the design elements throughout the visualization.

5. Contextualization

Provide context for the data being presented.

6. Accuracy

Ensure the visualization accurately represents the underlying data.

7. Visuals Encoding

Choose appropriate visual encodings for the data types you are visualizing.

8. Intuitiveness

Design the visualization to be intuitive and easy to comprehend.

9. Interactivity

Consider adding interactive elements to the visualization, such as tooltips, zooming, filtering, or highlighting.

10. Aesthetics

Although aesthetics are subjective, a visually appealing design can engage viewers and increase their interest in the data.

11. Accessibility

Accessibility is key; if users can’t read the data, it’s useless.

12. Hierarchy

Work out hierarchy of information early on and always remind yourself of what the purpose of representing the data is.

OR

1. Clarity and Simplicity

- **Principle**: Keep visualizations clear, straightforward, and focused on the main message.
- **Why It Matters**: A simple visualization ensures that the audience can quickly grasp the core insights without being distracted by unnecessary elements.
- **How to Achieve It**:
    - Avoid clutter—remove any non-essential elements like excessive gridlines, 3D effects, or irrelevant labels.
    - Stick to a clean design with minimalistic backgrounds and use whitespace effectively to improve readability.

2. Accurate Representation of Data

- **Principle**: Always present data in a truthful and accurate manner.
- **Why It Matters**: Misleading visualizations can distort the data's message, leading to incorrect conclusions and decisions.
- **How to Achieve It**:
    - Label axes and provide clear scale marks to ensure that data points are correctly represented.
    - Avoid manipulative practices, such as truncating axes, which could exaggerate or downplay trends.
    - Ensure the scale and proportions are consistent.

3. Choosing the Right Chart Type

- **Principle**: Select the most appropriate visualization method for the type of data and the insights you want to convey.
- **Why It Matters**: The correct chart type helps convey the message more effectively, while the wrong one can confuse the audience or obscure important insights.
- **How to Achieve It**:
    - **Bar charts** are best for comparing categorical data.
    - **Line charts** are ideal for showing trends over time.
    - **Pie charts** are useful for illustrating proportions.

- **Scatter plots** work well for showing relationships between two continuous variables.
- **Heatmaps** help visualize data patterns in matrices, such as correlations.

4. Context and Labeling

- **Principle**: Provide adequate context and label all axes, legends, and data points to clarify what the data represents.
- **Why It Matters**: Without context, viewers may misinterpret the data. Clear labeling helps the audience understand the meaning of the data quickly and accurately.
- **How to Achieve It**:
  - Use clear titles that explain the purpose of the visualization.
  - Label axes with both variable names and units of measurement.
  - Include a legend or key when using color coding or symbols.

5. Consistency

- **Principle**: Maintain consistency in design elements across all visualizations.
- **Why It Matters**: Consistent design improves understanding and reduces confusion, allowing the audience to focus on the data.
- **How to Achieve It**:
  - Use the same colors, fonts, and style for similar data points across different visualizations.
  - Ensure that the scales and axes are consistent across visualizations in the same dataset to facilitate comparison.

6. Use of Color

- **Principle**: Use color strategically to enhance the clarity of the visualization and highlight important data points.
- **Why It Matters**: Color can attract attention, differentiate data series, and make trends stand out, but it can also be distracting if used improperly.
- **How to Achieve It**:
  - Use contrasting colors to distinguish different categories or groups.
  - Stick to a limited color palette to avoid overwhelming the viewer.
  - Be mindful of color-blind users by using color schemes that are easily distinguishable for all audiences (e.g., using colorblind-friendly palettes).

7. Tell a Story with Data

- **Principle**: Create a visualization that guides the viewer through the data in a way that tells a clear, cohesive story.
- **Why It Matters**: Data science is not just about presenting numbers but about providing insights that lead to actionable conclusions. A storytelling approach engages the audience and ensures that the key message is delivered.
- **How to Achieve It**:
    - Structure the visualization so it leads the viewer from one insight to the next, building understanding progressively.
    - Use annotations to highlight important findings or trends.
    - Make sure that the data visualization supports the narrative you're trying to convey.

8. Effective Use of Space

- **Principle**: Use space wisely to avoid clutter and ensure readability.
- **Why It Matters**: Poor use of space can make visualizations difficult to read and interpret, especially when dealing with large datasets.
- **How to Achieve It**:
    - Ensure there’s enough space between data points, labels, and axes.
    - Adjust the layout so that visual elements are logically organized and easily follow the viewer’s eye.

9. Interactivity and Exploration

- **Principle**: Provide interactive elements when appropriate, allowing users to explore the data in more depth.
- **Why It Matters**: Interactive visualizations can help users better understand the data by enabling them to filter, zoom in, or hover over data points for additional details.
- **How to Achieve It**:
    - Use interactive tools like dropdown menus, clickable filters, and tooltips to allow users to explore the data.
    - Interactive dashboards can give users control over how they view data, making the experience more personalized and dynamic.

10. Avoid Overloading Information

- **Principle**: Avoid including too much data in a single visualization, as it can overwhelm or confuse the viewer.
- **Why It Matters**: Overloading a visualization with excessive information dilutes the key insights and makes it harder for the audience to extract meaning.

- **How to Achieve It**:
    - Limit the number of data series or categories to only the most important ones.
    - Break down complex datasets into smaller, more digestible visualizations if necessary.

11. Scalability and Responsiveness

- **Principle**: Ensure that the visualization is scalable and adaptable to different platforms and screen sizes.
- **Why It Matters**: In today's world, data visualizations are often viewed on various devices, from mobile phones to large screens. The design should adapt to ensure clarity and usability on any device.
- **How to Achieve It**:
    - Use responsive design principles for web-based visualizations.
    - Ensure text and labels are legible on smaller screens and consider simplifying the design for mobile use.

12. Accessibility

- **Principle**: Make visualizations accessible to a wide audience, including those with disabilities.
- **Why It Matters**: Ensuring accessibility ensures that your insights reach a broader audience and are not limited by visual impairments or other accessibility challenges.
- **How to Achieve It**:
    - Use color palettes that are friendly to color-blind individuals.
    - Provide textual descriptions or data summaries for key visual elements for those who may not be able to interpret the visualization effectively.

Data Visualization Tools and Libraries ,

1. Data Visualization Tools (GUI-Based Tools)

These are user-friendly tools that allow users to create visualizations without writing code. They are often used by business analysts, managers, and non-technical users.

a. Tableau

- **Overview**: A leading business intelligence tool used for data visualization and dashboarding.
- **Key Features**:
    - Drag-and-drop interface for creating a wide range of charts and dashboards.
    - Supports interactive dashboards and real-time data updates.

    - Integrates with various data sources (databases, cloud services, etc.).
    - Powerful features for filtering, grouping, and aggregating data.
- **Best For**: Business professionals who need to visualize data quickly and create shareable dashboards.

b. Power BI

- **Overview**: A Microsoft tool designed for business intelligence and data visualization.
- **Key Features**:
    - Integrates with Microsoft Excel, SQL Server, and other databases.
    - Provides a rich set of visualization types including charts, maps, and KPI indicators.
    - Supports custom visualizations and interactive reporting.
- **Best For**: Users within the Microsoft ecosystem (Excel, Azure, etc.) who want interactive dashboards and reports.

c. Qlik Sense

- **Overview**: A self-service data visualization tool with interactive dashboards and analytical capabilities.
- **Key Features**:
    - Supports data exploration and visualization through a drag-and-drop interface.
    - Associative data model for connecting multiple data sources seamlessly.
    - Built-in AI features for insights and recommendations.
- **Best For**: Organizations needing data exploration with flexible visualizations.

d. Google Data Studio

- **Overview**: A free tool by Google to create reports and dashboards with interactive data visualizations.
- **Key Features**:
    - Easy integration with Google Analytics, Google Ads, and Google Sheets.
    - Customizable templates and drag-and-drop features.
    - Allows collaborative report creation.
- **Best For**: Teams and individuals using Google products looking for an easy, cost-effective solution.

e. Domo

- **Overview**: A cloud-based platform for business intelligence and data visualization.
- **Key Features**:
    - Provides a wide range of pre-built connectors and integrations for real-time data access.
    - Offers both visualization and data preparation features.
    - Customizable dashboards and reports with real-time collaboration features.
- **Best For**: Large enterprises with diverse data sources looking for real-time dashboards.

These libraries are so popular because they allow analysts and statisticians to create visual data models easily according to their specifications by conveniently providing an

interface, and data visualization tools all in one place! The list explains some key features and applications that are being used to prepare data for visualization.

Libraries:

1. Matplotlib

Matplotlib is a data visualization library and 2-D plotting library of Python It was initially released in 2003 and it is the most popular and widely-used plotting library in the Python community. It comes with an interactive environment across multiple platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, etc. It can be used to embed plots into applications using various GUI toolkits like Tkinter, GTK+, wxPython, Qt, etc. So you can use Matplotlib to create plots, bar charts, pie charts, histograms, scatterplots, error charts, power spectra, stemplots, and whatever other visualization charts you want! **The Pyplot module also provides a MATLAB-like interface that is just as versatile and useful as MATLAB while being free and open source.**

2. Plotly

Plotly is a free open-source graphing library that can be used to form data visualizations. Plotly (plotly.py) is built on top of the Plotly JavaScript library (plotly.js) and can be used to create web-based data visualizations that can be displayed in Jupyter notebooks or web applications using Dash or saved as individual HTML files. Plotly provides more than 40 unique chart types like scatter plots, histograms, line charts, bar charts, pie charts, error bars, box plots, multiple axes, sparklines, dendrograms, 3-D charts, etc. Plotly also provides contour plots, which are not that common in other data visualization libraries. In addition to all this, Plotly can be used offline with no internet connection.

3. Seaborn

Seaborn is a Python data visualization library that is based on Matplotlib and closely integrated with the NumPy and pandas data structures. Seaborn has various dataset-oriented plotting functions that operate on data frames and arrays that have whole datasets within them. Then it internally performs the necessary statistical aggregation and mapping functions to create informative plots that the user desires. It is a high-level interface for creating beautiful and informative statistical graphics that are integral to exploring and understanding data. The Seaborn data graphics can include bar charts, pie charts, histograms, scatterplots, error charts, etc. Seaborn also has various tools for choosing colour palettes that can reveal patterns in the data.

4. GGplot

Ggplot is a Python data visualization library that is based on the implementation of ggplot2 which is created for the programming language R. Ggplot can create data visualizations such as bar charts, pie charts, histograms, scatterplots, error charts, etc. using high-level API. It also allows you to add different types of data visualization components or layers in a single visualization. Once ggplot has been told which variables to map to which aesthetics in the plot, it does the rest of the work so that the user can focus on interpreting the visualizations and take less time in creating them. But this also means that it is not possible to

create highly customized graphics in ggplot. **Ggplot is also deeply connected with pandas so it is best to keep the data in DataFrames.**

5. Altair

Altair is a statistical data visualization library in Python. **It is based on Vega and Vega-Lite which are a sort of declarative language for creating, saving, and sharing data visualization designs that are also interactive**. Altair can be used to create beautiful data visualizations of plots such as bar charts, pie charts, histograms, scatterplots, error charts, power spectra, stemplots, etc. using a minimal amount of coding. Altair has dependencies which include Python 3.6, entry points, jsonschema, NumPy, Pandas, and Toolz which are automatically installed with the Altair installation commands. You can open Jupyter Notebook or JupyterLab and execute any of the code to obtain those data visualizations in Altair. Currently, the source for Altair is available on GitHub.

6. Bokeh

Bokeh is a data visualization library that provides detailed graphics with a high level of interactivity across various datasets, whether they are large or small. Bokeh is based on The Grammar of Graphics like ggplot but it is native to Python while ggplot is based on ggplot2 from R. Data visualization experts can create various interactive plots for modern web browsers using bokeh which can be used in interactive web applications, HTML documents, or JSON objects. Bokeh has 3 levels that can be used for creating visualizations. The first level focuses only on creating the data plots quickly, the second level controls the basic building blocks of the plot while the third level provides full autonomy for creating the charts with no pre-set defaults. This level is suited to data analysts and IT professionals that are well-versed in the technical side of creating data visualizations.

7. Geoplotlib

Most of the data visualization libraries don't provide much support for creating maps or using geographical data and that is why geoplotlib is such an important Python library. It supports the creation of geographical maps in particular with many different types of maps available such as dot-density maps, choropleths, symbol maps, etc. One thing to keep in mind is that requires NumPy and pyglet as prerequisites before installation but that is not a big disadvantage. Especially since you want to create geographical maps and geoplotlib is the only excellent option for maps out there!

Python Libraries: Matplotlib: Basic Plotting (Histograms, Box Plots, Scatter Plots, Identifying Patterns, Trends, and Outliers) ,

Data visualization is a crucial aspect of data analysis, enabling data scientists and analysts to present complex data in a more understandable and insightful manner. One of the most popular libraries for data visualization in Python is Matplotlib. In this article, we will provide a comprehensive guide to using Matplotlib for creating various types of plots and customizing them to fit specific needs and how to visualize data with the help of the Matplotlib library of Python.

Installing Matplotlib for Data Visualization

We will use the pip command to install this module. If you do not have pip installed then refer to the article, **Download and install pip Latest Version.**

To install Matplotlib type the below command in the terminal.

pip install matplotlib

If you are using Jupyter Notebook, you can install it within a notebook cell by using:

```
!pip install matplotlib
```

Data Visualization With Pyplot in Matplotlib

Matplotlib provides a module called pyplot, which offers a MATLAB-like interface for creating plots and charts. It simplifies the process of generating various types of visualizations by offering a collection of functions that handle common plotting tasks. Each function in Pyplot modifies a figure in a specific way, such as:

Key Pyplot Functions:

Category	Function	Description
Plot Creation	plot()	Creates line plots with customizable styles.
	scatter()	Generates scatter plots to visualize relationships.
Graphical Elements	bar()	Creates bar charts for comparing categories.
	hist()	Draws histograms to show data distribution.
	pie()	Creates pie charts to represent parts of a whole.
Customization	xlabel(), ylabel()	Sets labels for the X and Y axes.
	title()	Adds a title to the plot.
	legend()	Adds a legend to differentiate data series.
Visualization	xlim(), ylim()	Sets limits for the X and Y axes.

Category	Function	Description
Control		
	grid()	Adds gridlines to the plot for readability.
	show()	Displays the plot in a window.
Figure Management	figure()	Creates or activates a figure.
	subplot()	Creates a grid of subplots within a figure.
	savefig()	Saves the current figure to a file.

Basic Plotting with Matplotlib : Step-by-Step Examples

Matplotlib supports a variety of plots including line charts, bar charts, histograms, scatter plots, etc. We will discuss the most commonly used charts in this article with the help of some good examples and will also see how to customize each plot.

1. Line Chart

Line chart is one of the basic plots and can be created using the **plot()** function. It is used to represent a relationship between two data X and Y on a different axis.

Syntax:

*matplotlib.pyplot.plot(*args, scalex=True, scaley=True, data=None, **kwargs)*

Example:

```
import matplotlib.pyplot as plt

# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

# plotting the data
plt.plot(x, y)

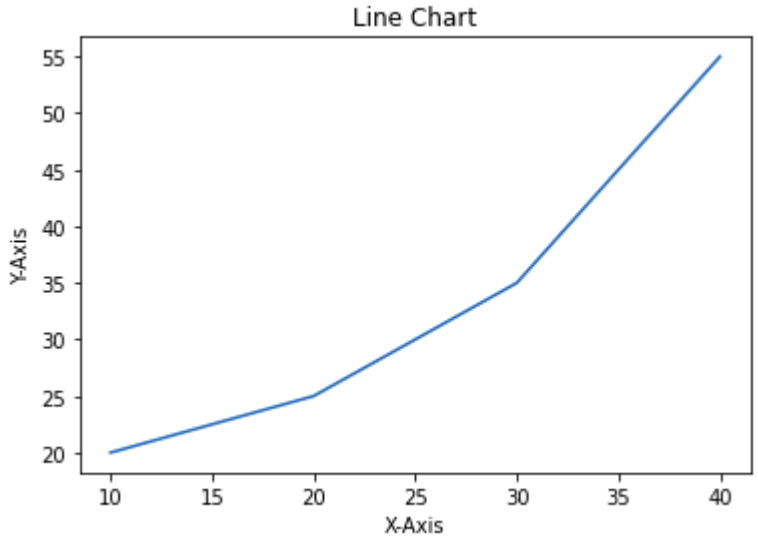
# Adding title to the plot
plt.title("Line Chart")

# Adding label on the y-axis
plt.ylabel('Y-Axis')

# Adding label on the x-axis
plt.xlabel('X-Axis')
```

plt.show()

Output:



2. Bar Chart

A bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent. The bar plots can be plotted horizontally or vertically. A bar chart describes the comparisons between the discrete categories. It can be created using the bar() method.

In the below example, we will use the tips dataset. Tips database is the record of the tip given by the customers in a restaurant for two and a half months in the early 1990s. It contains 6 columns as total_bill, tip, sex, smoker, day, time, size.

Example:

```
import matplotlib.pyplot as plt

import pandas as pd


# Reading the tips.csv file

data = pd.read_csv('tips.csv')


# initializing the data

x = data['day']

y = data['total_bill']


# plotting the data

plt.bar(x, y)
```

```
# Adding title to the plot

plt.title("Tips Dataset")


# Adding label on the y-axis

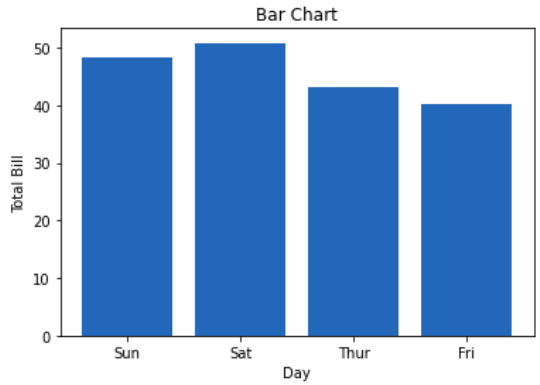
plt.ylabel('Total Bill')


# Adding label on the x-axis

plt.xlabel('Day')

plt.show()
```

Output:



3. Histogram

A **histogram** is basically used to represent data provided in a form of some groups. It is a type of bar plot where the X-axis represents the bin ranges while the Y-axis gives information about frequency. The **hist()** function is used to compute and create histogram of x.

Syntax:

*matplotlib.pyplot.hist(x, bins=None, range=None, density=False, weights=None, cumulative=False, bottom=None, histtype='bar', align='mid', orientation='vertical', rwidth=None, log=False, color=None, label=None, stacked=False, *, data=None, ***kwargs)*

Example:

```
import matplotlib.pyplot as plt
import pandas as pd


# Reading the tips.csv file
data = pd.read_csv('tips.csv')


# initializing the data
x = data['total_bill']
```

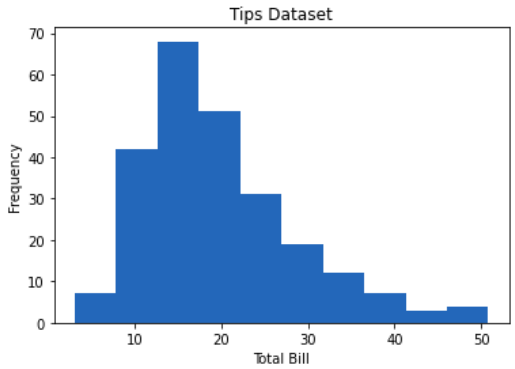
```
# plotting the data
plt.hist(x)


# Adding title to the plot
plt.title("Tips Dataset")


# Adding label on the y-axis
plt.ylabel('Frequency')


# Adding label on the x-axis
plt.xlabel('Total Bill')
plt.show()
```

Output:



4. Scatter Plot

Scatter plots are used to observe relationships between variables. The **scatter()** method in the matplotlib library is used to draw a scatter plot.

Syntax:

matplotlib.pyplot.scatter(x_axis_data, y_axis_data, s=None, c=None, marker=None, cmap=None, vmin=None, vmax=None, alpha=None, linewidths=None, edgecolors=None

Example:

```
import matplotlib.pyplot as plt
import pandas as pd


# Reading the tips.csv file
data = pd.read_csv('tips.csv')


# initializing the data
x = data['day']
y = data['total_bill']

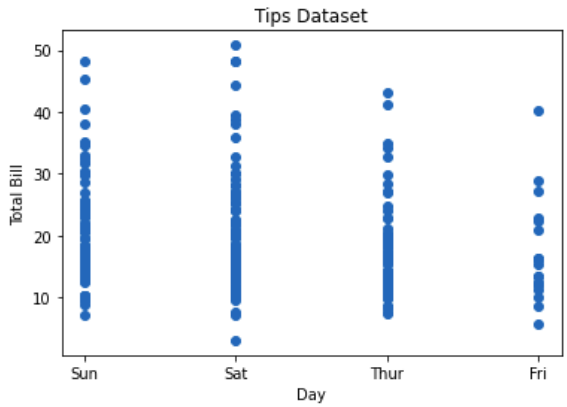

# plotting the data
plt.scatter(x, y)
```

```
# Adding title to the plot
plt.title("Tips Dataset")


# Adding label on the y-axis
plt.ylabel('Total Bill')


# Adding label on the x-axis
plt.xlabel('Day')
plt.show()
```

Output:



5. Pie Chart

Pie chart is a circular chart used to display only one series of data. The area of slices of the pie represents the percentage of the parts of the data. The slices of pie are called wedges. It can be created using the pie() method.

Syntax:

matplotlib.pyplot.pie(data, explode=None, labels=None, colors=None, autopct=None, shadow=False)

Example:

```
import matplotlib.pyplot as plt
import pandas as pd

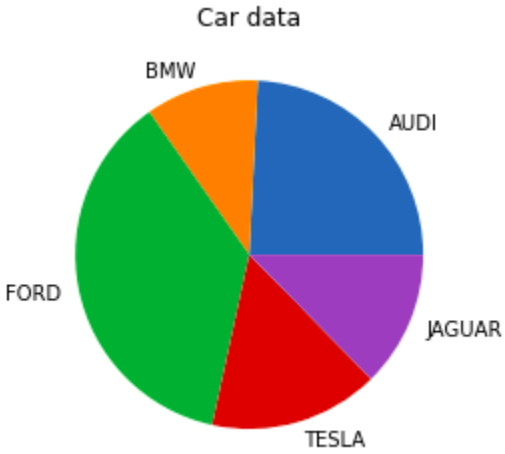

# Reading the tips.csv file
data = pd.read_csv('tips.csv')


# initializing the data
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR',]
data = [23, 10, 35, 15, 12]


# plotting the data
plt.pie(data, labels=cars)
```

```
# Adding title to the plot
plt.title("Car data")
plt.show()
```

Output:



6. Box Plot

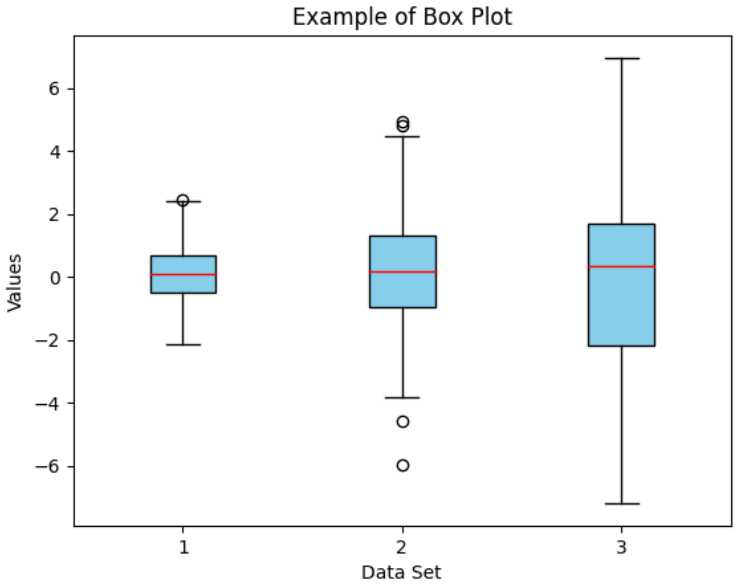
A Box Plot, also known as a Whisker Plot, is a standardized way of displaying the distribution of data based on a five-number summary: minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum. It can also show outliers.
Let's see an example of how to create a Box Plot using Matplotlib in Python:

Example:

```
import matplotlib.pyplot as plt
import numpy as np
np.random.seed(10)
data = [np.random.normal(0, std, 100) for std in range(1, 4)]


# Create a box plot
plt.boxplot(data, vert=True, patch_artist=True, boxprops=dict(facecolor='skyblue'),
medianprops=dict(color='red'))
plt.xlabel('Data Set')
plt.ylabel('Values')
plt.title('Example of Box Plot')
plt.show()
```

Output:



Box Plot in Python using Matplotlib

Explanation:

- **plt.boxplot(data)**: Creates the box plot**. The vert=True argument makes the plot vertical, and patch_artist=True fills the box with color.**
- **boxprops and medianprops**: Customize the appearance of the boxes and median lines, respectively.

The box shows the interquartile range (IQR), the line inside the box shows the median, and the "whiskers" extend to the minimum and maximum values within 1.5 * IQR from the first and third quartiles. Any points outside this range are considered outliers and are plotted as individual points.

7. Heatmap

A Heatmap is a data visualization technique that represents data in a matrix form, where individual values are represented as colors. Heatmaps are particularly useful for visualizing the magnitude of data across a two-dimensional surface and identifying patterns, correlations, and concentrations.

Let's see an example of how to create a Heatmap using Matplotlib in Python:

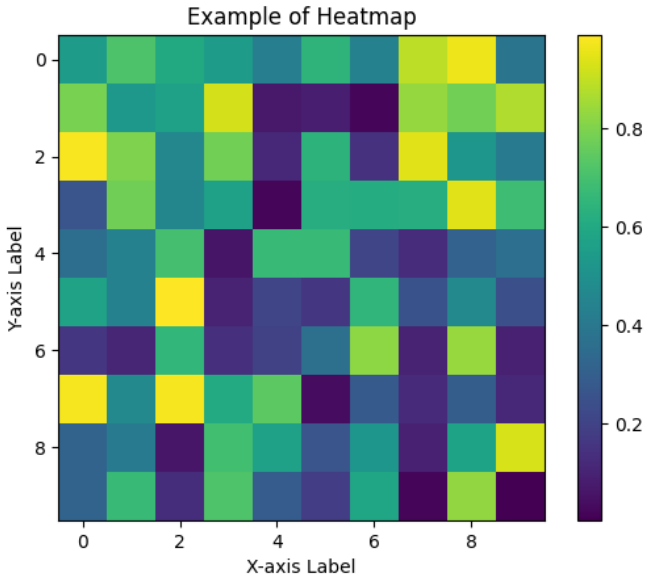
Example:

```
import matplotlib.pyplot as plt
import numpy as np
np.random.seed(0)
data = np.random.rand(10, 10)


# Create a heatmap
plt.imshow(data, cmap='viridis', interpolation='nearest')


# Add a color bar to show the scale
plt.colorbar()
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Example of Heatmap')
plt.show()
```

Output:



Heatmap with matplotlib

Explanation:

- **plt.imshow(data, cmap='viridis')**: Displays the data as an image (heatmap). The cmap='viridis' argument specifies the color map used for the heatmap.
- **interpolation='nearest'**: Ensures that each data point is shown as a block of color without smoothing.

The color bar on the side provides a scale to interpret the colors, with darker colors representing lower values and lighter colors representing higher values. This type of plot is often used in fields like data analysis, bioinformatics, and finance to visualize data correlations and distributions across a matrix.

Seaborn: Statistical Plots, Heatmaps,

Seaborn is a Python data visualization library that simplifies the process of creating complex visualizations. ***It is specifically designed for statistical data visualization, making it easier to understand data distributions and relationships between variables***. Seaborn integrates closely with Pandas data structures, allowing seamless data manipulation and visualization.

It is built on the top of matplotlib library and also closely integrated into the data structures from pandas.

Key Features of Seaborn:

- **High-level interface**: Simplifies the creation of complex visualizations.
- **Integration with Pandas**: Works seamlessly with Pandas DataFrames for data manipulation.
- **Built-in themes**: Offers attractive default themes and color palettes.
- **Statistical plots**: Provides various plot types to visualize statistical relationships and distributions.

Installing Seaborn for Data Visualization

Before using Seaborn, you'll need to install it. The easiest way to install Seaborn is using pip, the Python package manager for python environment:

```
pip install seaborn
```

Creating Basic Plots with Seaborn

Before starting let's have a small intro of bivariate and univariate data:

- **Bivariate data:** This type of data involves **two different variables**. The analysis of this type of data deals with causes and relationships and the analysis is done to find out the relationship between the two variables.
- **Univariate data:** This type of data consists of **only one variable**. The analysis of univariate data is thus the simplest form of analysis since the information deals with only one quantity that changes. It does not deal with causes or relationships and the main purpose of the analysis is to describe the data and find patterns that exist within it.

Seaborn offers a variety of plot types to visualize different aspects of data. ***Seaborn helps to visualize the statistical relationships, to understand how variables in a dataset are related to one another and how that relationship is dependent on other variables, we perform statistical analysis.*** This Statistical analysis helps to visualize the trends and identify various patterns in the dataset. Below are some common plots you can create using Seaborn:

1. Line plot

Lineplot is the most popular plot to draw a relationship between x and y with the possibility of several semantic groupings. It is often used to track changes over intervals.

Syntax : *sns.lineplot(x=None, y=None)*

Parameters:

x, y: *Input data variables; must be numeric. Can pass data directly or reference columns in data.*

Let's visualize the data with a line plot and pandas:

Creating Basic Plots with Seaborn

Before starting let's have a small intro of bivariate and univariate data:

- **Bivariate data:** This type of data involves **two different variables**. The analysis of this type of data deals with causes and relationships and the analysis is done to find out the relationship between the two variables.
- **Univariate data:** This type of data consists of **only one variable**. The analysis of univariate data is thus the simplest form of analysis since the information deals with only one quantity that changes. It does not deal with causes or relationships and the main purpose of the analysis is to describe the data and find patterns that exist within it.

Seaborn offers a variety of plot types to visualize different aspects of data. ***Seaborn helps to visualize the statistical relationships, to understand how variables in a dataset are related to one another and how that relationship is dependent on other variables, we perform statistical analysis.*** This Statistical analysis helps to visualize the trends and identify various patterns in the dataset. Below are some common plots you can create using Seaborn:

1. Line plot

Lineplot is the most popular plot to draw a relationship between x and y with the possibility of several semantic groupings. It is often used to track changes over intervals.

Syntax : *sns.lineplot(x=None, y=None)*

Parameters:

x, y: *Input data variables; must be numeric. Can pass data directly or reference columns in data.*

Let's visualize the data with a line plot and pandas:

```
import pandas as pd

import seaborn as sns
```

```
# initialise data of lists

data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],

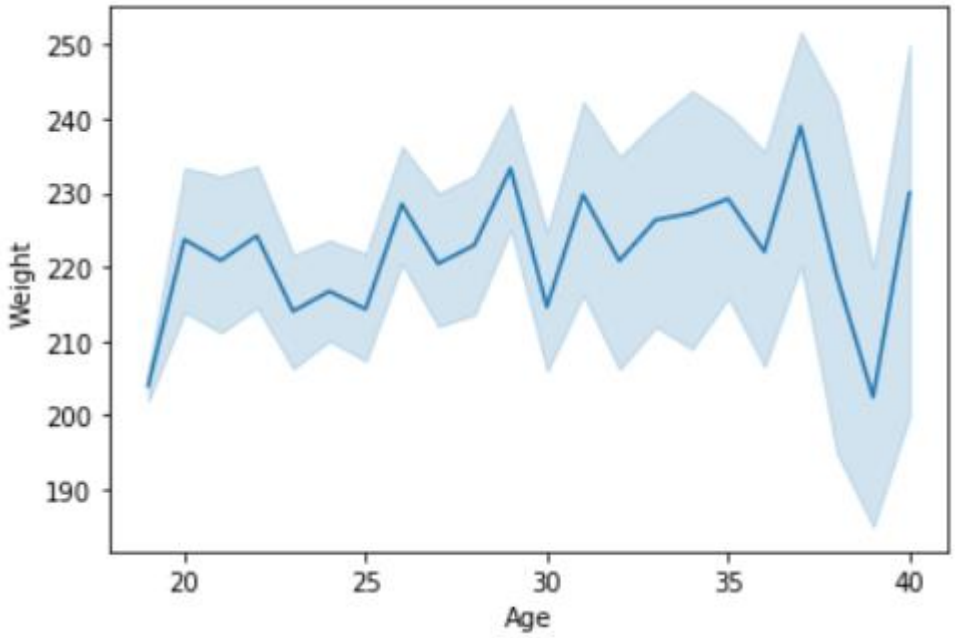
        'Age':[ 30 , 21 , 29 , 28 ]}

df = pd.DataFrame( data )


# plotting lineplot

sns.lineplot( data['Age'], data['Weight'])
```

Output:



2. Scatter Plot

Scatter plots are used to visualize the relationship between two numerical variables. They help identify correlations or patterns. It can draw a two-dimensional graph.

Syntax: *seaborn.scatterplot(x=None, y=None)*

Parameters:
x, y: *Input data variables that should be numeric.*
Returns: *This method returns the Axes object with the plot drawn onto it.*

Let's visualize the data with a scatter plot and pandas:

```
import pandas as pd
```

```
import seaborn as sns


# initialise data of lists

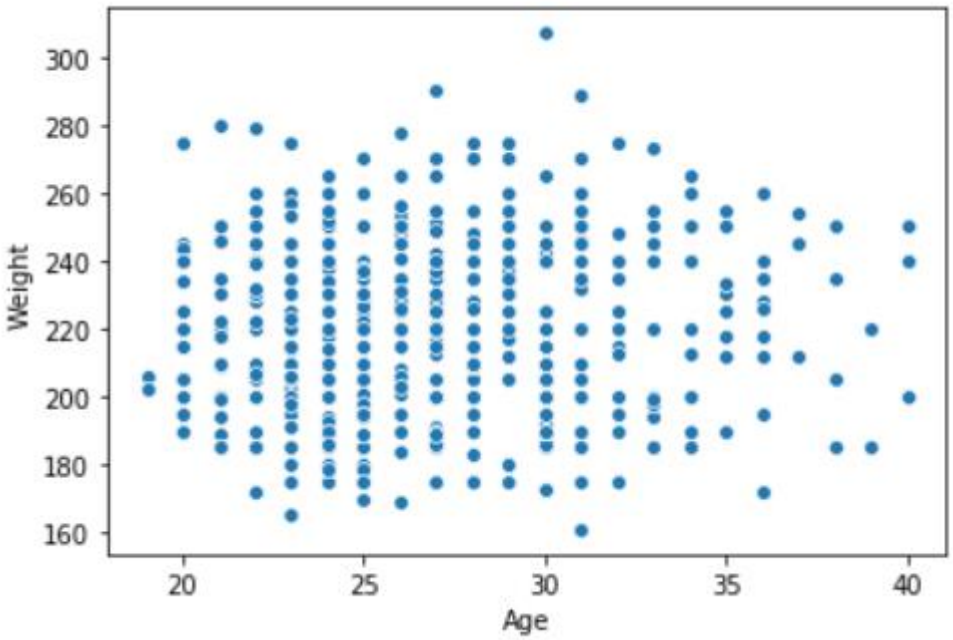
data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],

        'Age':[ 30 , 21 , 29 , 28 ]}

df = pd.DataFrame( data )


seaborn.scatterplot(data['Age'],data['Weight'])
```

Output:



3. Box plot

A box plot (or box-and-whisker plot) s is the visual representation of the depicting groups of numerical data through their quartiles against continuous/categorical data.

A box plot consists of 5 things.

- Minimum
- First Quartile or 25%
- Median (Second Quartile) or 50%
- Third Quartile or 75%
- Maximum

Syntax:

seaborn.boxplot(x=None, y=None, hue=None, data=None)

Parameters:

- ***x, y, hue:*** *Inputs for plotting long-form data.*
- ***data:*** *Dataset for plotting. If x and y are absent, this is interpreted as wide-form.*

Returns: *It returns the Axes object with the plot drawn onto it.*
Let's create box plot with seaborn with an example.

```
import pandas as pd

import seaborn as sns


# initialise data of lists

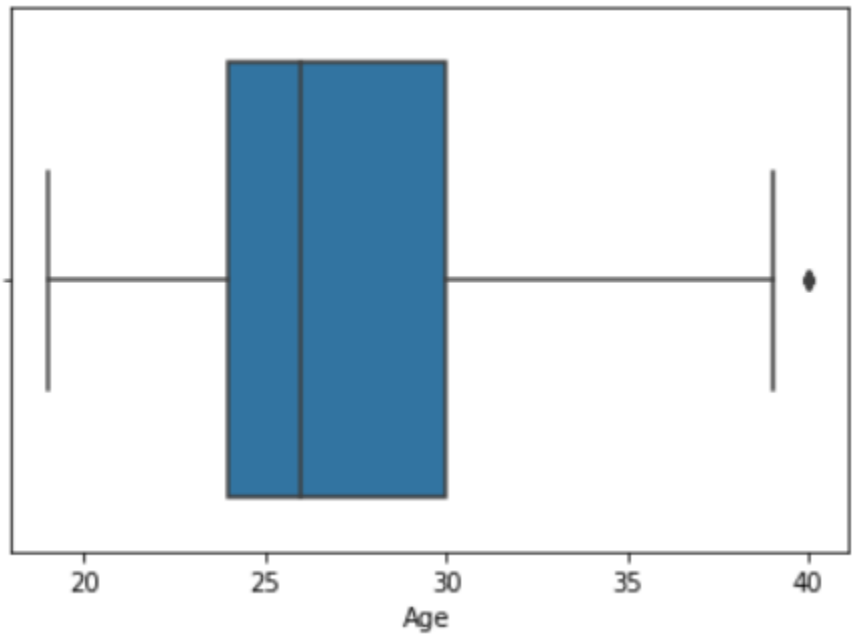
data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],

      'Age':[ 30 , 21 , 29 , 28 ]}

df = pd.DataFrame( data )

sns.boxplot( data['Age'] )
```

Output:



4. Violin Plot

A violin plot is similar to a boxplot. It shows several quantitative data across one or more categorical variables such that those distributions can be compared.
Syntax: *seaborn.violinplot(x=None, y=None, hue=None, data=None)*
Parameters:

- ***x, y, hue:*** *Inputs for plotting long-form data.*
- ***data:*** *Dataset for plotting.*

Example 1: Draw the violin plot with Pandas

```
import pandas as pd

import seaborn as sns


# initialise data of lists

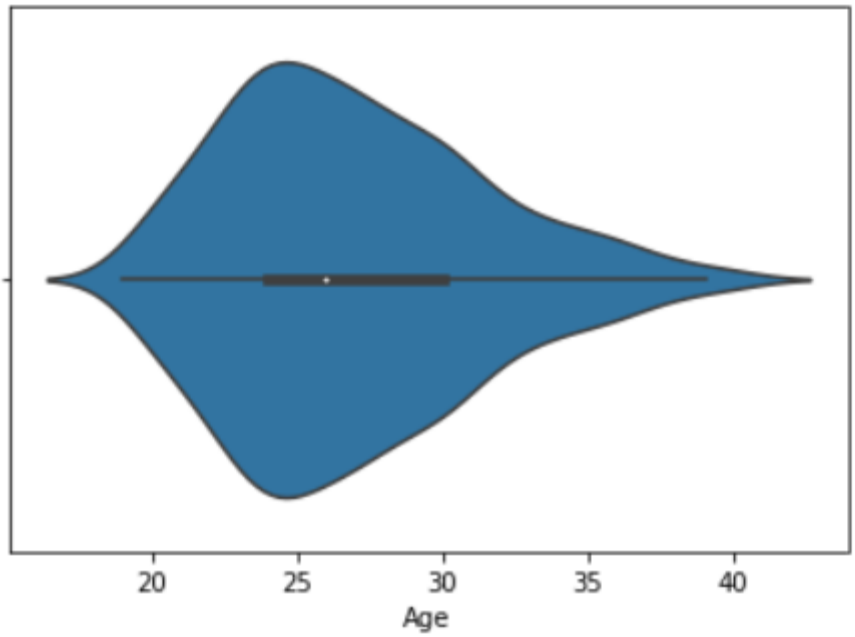
data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ]

      'Age':[ 30 , 21 , 29 , 28 ]}

df = pd.DataFrame( data )

sns.violinplot(data['Age'])
```

Output:



5. Swarm plot

A swarm plot is similar to a strip plot, We can draw a swarm plot with non-overlapping points against categorical data.

Syntax: *seaborn.swarmplot(x=None, y=None, hue=None, data=None)*

Parameters:

- ***x, y, hue:*** *Inputs for plotting long-form data.*
- ***data:*** *Dataset for plotting.*

Example : Draw the swarm plot with Pandas

```
# import module

import seaborn

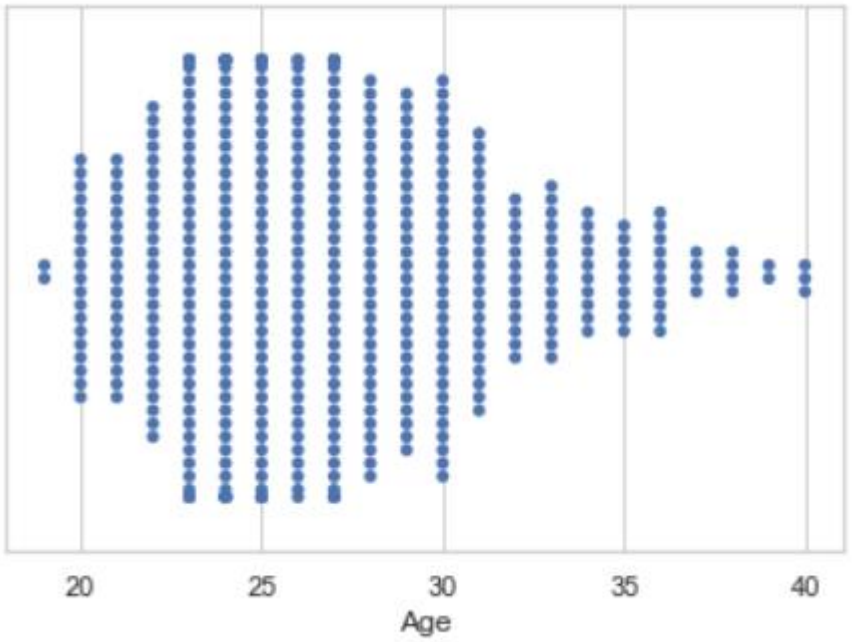

seaborn.set(style = 'whitegrid')


# read csv and plot

data = pandas.read_csv( "nba.csv" )


seaborn.swarmplot(x = data["Age"])
```

Output:



6. Bar plot

Barplot represents an estimate of central tendency for a numeric variable with the height of each rectangle and provides some indication of the uncertainty around that estimate using error bars.

Syntax : *seaborn.barplot(x=None, y=None, hue=None, data=None)*

Parameters :

- ***x, y :*** *This parameter take names of variables in data or vector data, Inputs for plotting long-form data.*
- ***hue :*** *(optional) This parameter take column name for colour encoding.*
- ***data :*** *(optional) This parameter take DataFrame, array, or list of arrays, Dataset for plotting. If x and y are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.*

Returns : *Returns the Axes object with the plot drawn onto it.*

Example : Draw the bar plot with Pandas

```
# import module

import seaborn

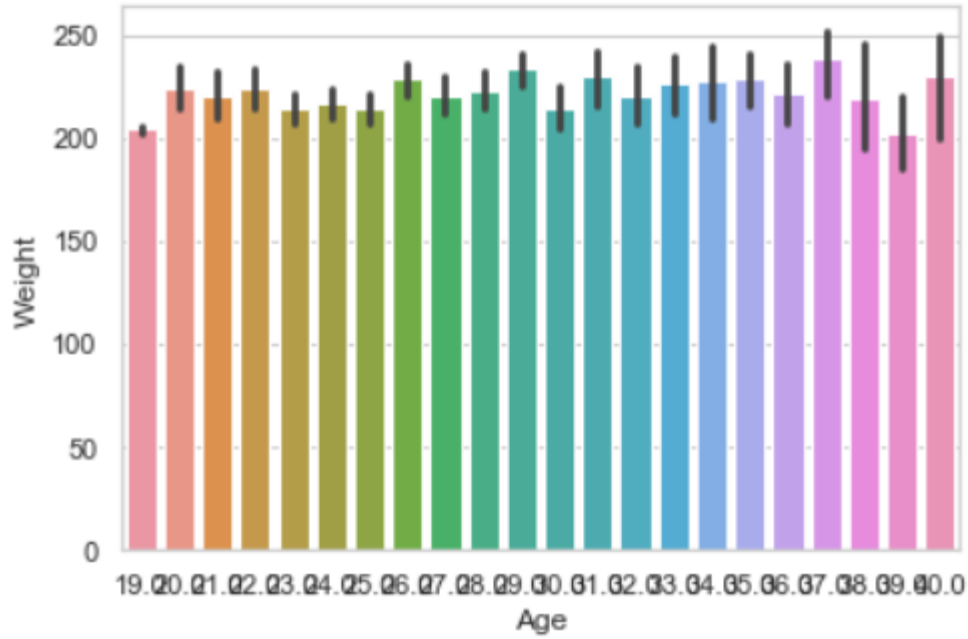

seaborn.set(style = 'whitegrid')


# read csv and plot

data = pandas.read_csv("nba.csv")


seaborn.barplot(x ="Age", y ="Weight", data = data)
```

Output:



7. Point plot

Point plot used to show point estimates and confidence intervals using scatter plot glyphs. A point plot represents an estimate of central tendency for a numeric variable by the position of scatter plot points and provides some indication of the uncertainty around that estimate using error bars.

Syntax: *seaborn.pointplot(x=None, y=None, hue=None, data=None)*

Parameters:

- ***x, y:*** *Inputs for plotting long-form data.*
- ***hue:*** *(optional) column name for color encoding.*
- ***data:*** *dataframe as a Dataset for plotting.*

Return: *The Axes object with the plot drawn onto it.*

Example: Draw the point plot with Pandas

```
# import module

import seaborn

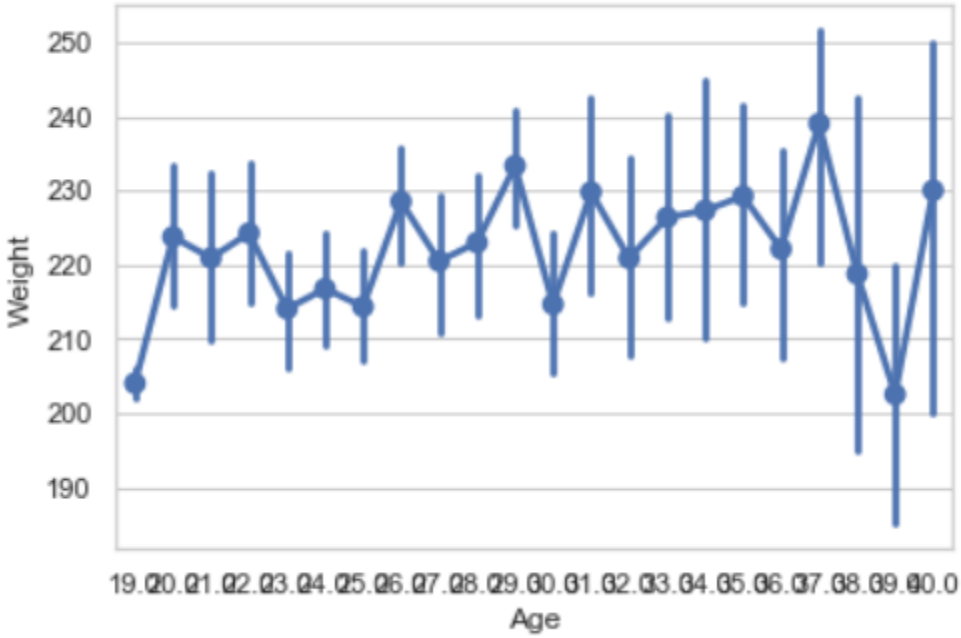

seaborn.set(style = 'whitegrid')


# read csv and plot

data = pandas.read_csv("nba.csv")


seaborn.pointplot(x = "Age", y = "Weight", data = data)
```

Output:



8. Count plot

Count plot used to Show the counts of observations in each categorical bin using bars.

Syntax : *seaborn.countplot(x=None, y=None, hue=None, data=None)*

Parameters :

- ***x, y:*** *This parameter take names of variables in data or vector data, optional, Inputs for plotting long-form data.*
- ***hue :*** *(optional) This parameter take column name for color encoding.*
- ***data :*** *(optional) This parameter take DataFrame, array, or list of arrays, Dataset for plotting. If x and y are absent, this is interpreted as wide-form. Otherwise, it is expected to be long-form.*

Returns: *Returns the Axes object with the plot drawn onto it.*

Example: Draw the count plot with Pandas

```
# import module

import seaborn

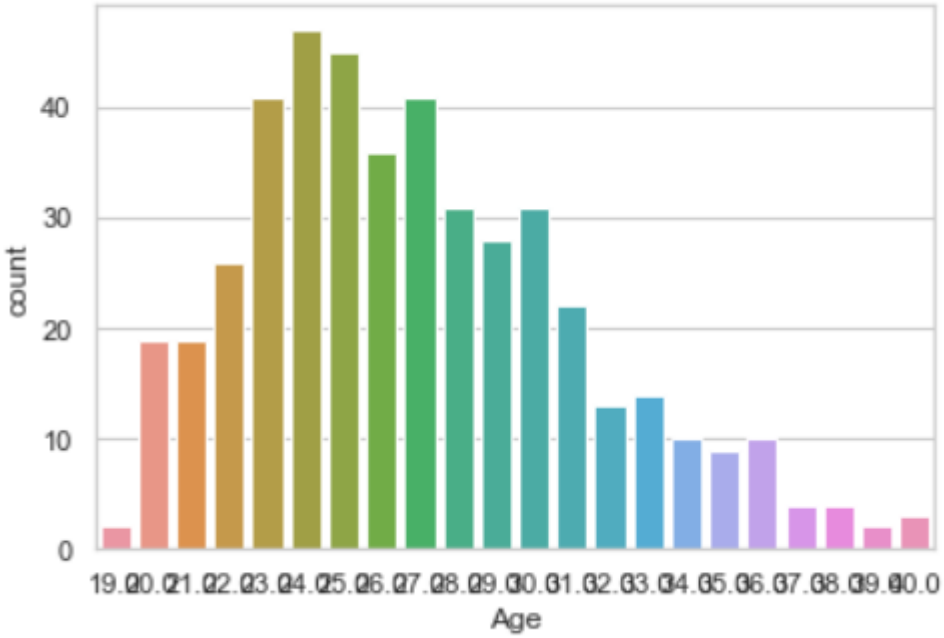

seaborn.set(style = 'whitegrid')


# read csv and plot

data = pandas.read_csv("nba.csv")


seaborn.countplot(data["Age"])
```

Output:



9. KDE Plot

KDE Plot described as **Kernel Density Estimate** is used for visualizing the Probability Density of a continuous variable. It depicts the probability density at different values in a continuous variable. We can also plot a single graph for multiple samples which helps in more efficient data visualization.

Syntax: *seaborn.kdeplot(x=None, *, y=None, vertical=False, palette=None, **kwargs)*

Parameters:

x, y : *vectors or keys in data*

vertical : *boolean (True or False)*

data : *pandas.DataFrame, numpy.ndarray, mapping, or sequence*

Example : Draw the KDE plot with Pandas

```
# importing the required libraries

from sklearn import datasets

import pandas as pd

import seaborn as sns


# Setting up the Data Frame

iris = datasets.load_iris()


iris_df = pd.DataFrame(iris.data, columns=['Sepal_Length',

                    'Sepal_Width', 'Patal_Length', 'Petal_Width'])


iris_df['Target'] = iris.target

  iris_df['Target'].replace([0], 'Iris_Setosa', inplace=True)

iris_df['Target'].replace([1], 'Iris_Vercicolor', inplace=True)

iris_df['Target'].replace([2], 'Iris_Virginica', inplace=True)

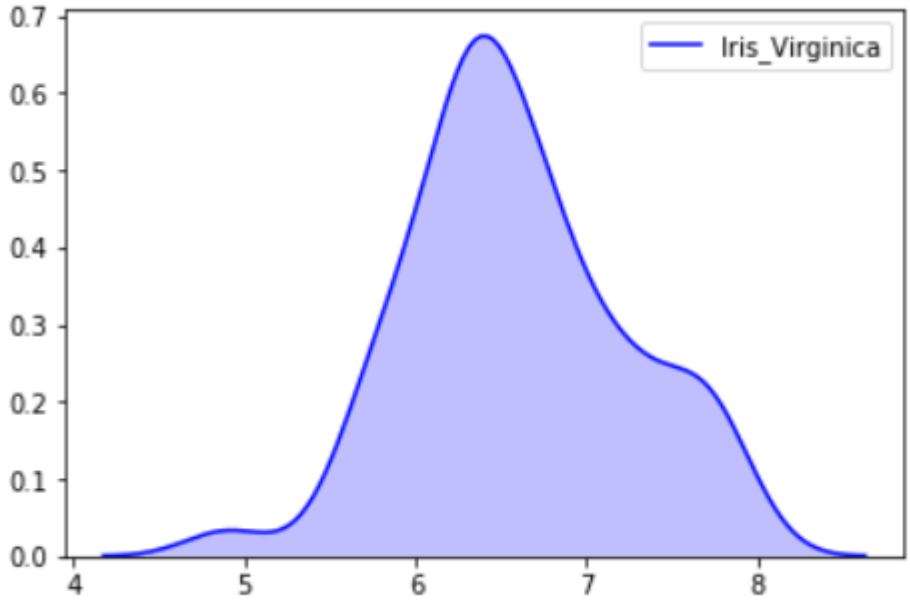


# Plotting the KDE Plot

sns.kdeplot(iris_df.loc[(iris_df['Target'] =='Iris_Virginica'),

          'Sepal_Length'], color = 'b', shade = True, Label ='Iris_Virginica')
```

Output:



Heatmaps:

Seaborn is a powerful Python library for data visualization, built on top of Matplotlib. One of its most popular features is the heatmap, which provides a visual representation of data in a matrix format. Adding a title to a Seaborn heatmap can enhance the readability and interpretability of the plot. This article will guide you through various methods to add a title to a Seaborn heatmap, along with examples and best practices.

Different Techniques for Adding Title to Seaborn Heatmap

Adding a title to your Seaborn heatmap is a straightforward process. After creating heatmap using the sns.heatmap() function, **use Matplotlib's plt.title() function to add a title.** This function allows to specify the title text and customize its appearance, such as font size and color.
For the implementation, we will use below dataframe:

```
import seaborn as sns

import matplotlib.pyplot as plt

import pandas as pd


# Sample data

data = pd.DataFrame({
```

```
    'A': [1, 2, 3, 4],

    'B': [4, 3, 2, 1],

    'C': [1, 3, 2, 4]

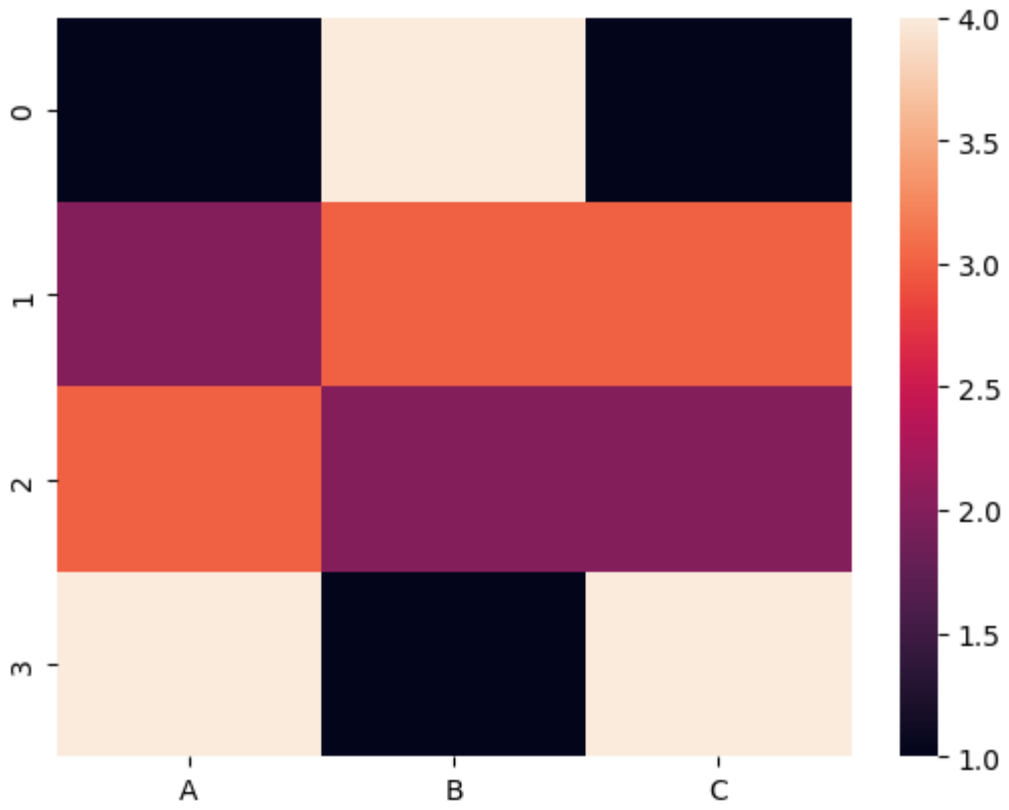
})


# Create heatmap

sns.heatmap(data)


plt.show()
```

Output:



Seaborn heatmap

1. Adding a Title Using Matplotlib

The most straightforward way to add a title to a Seaborn heatmap is by using Matplotlib's `plt.title()` function. This method is simple and effective for most use cases.

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns

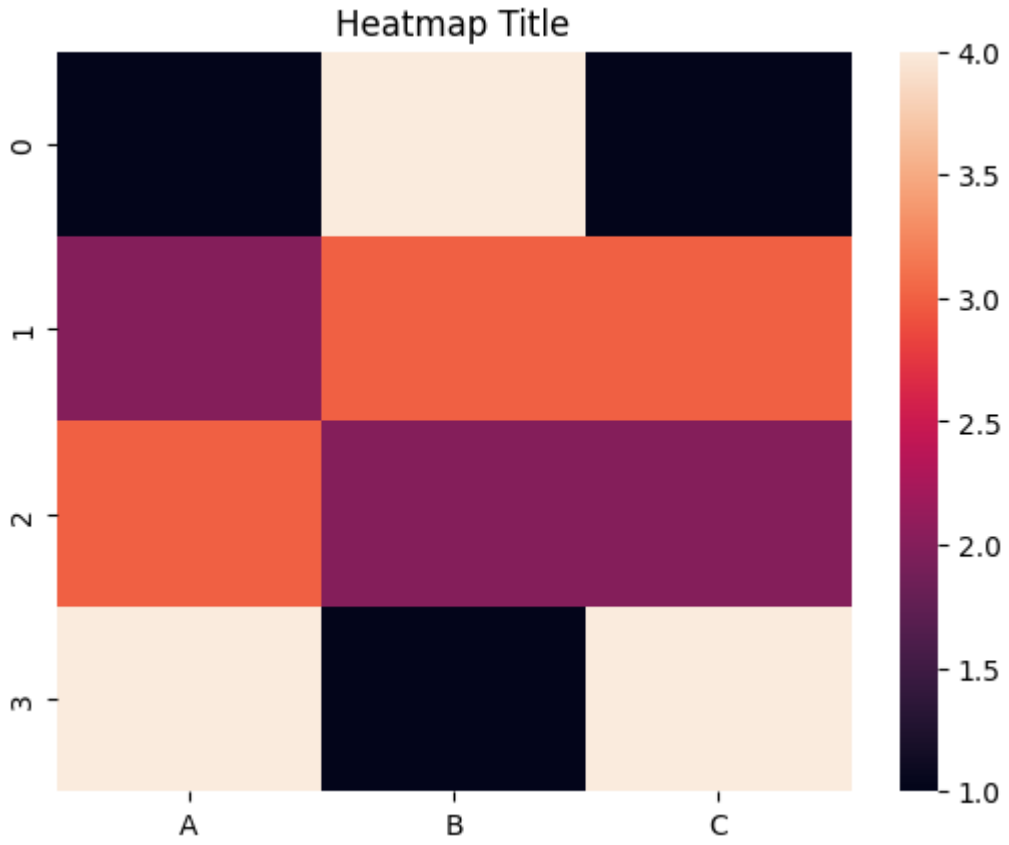

# Create heatmap

sns.heatmap(data)


# Add title

plt.title('Heatmap Title')

plt.show()
```
Output:



Adding a Title Using Matplotlib

2. Using Seaborn's `set_title` Method

Seaborn provides a method called `set_title()` that can be used to add a title directly to the Axes object returned by the `heatmap()` function.

```
import seaborn as sns
```

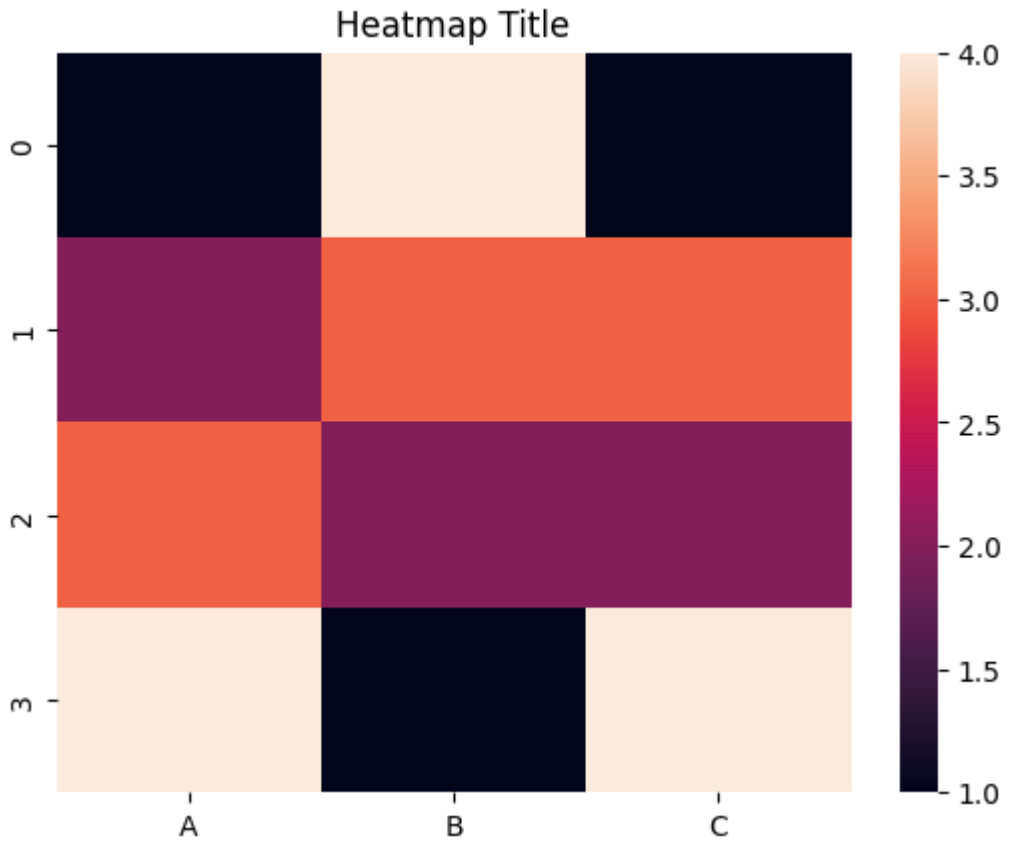
```
# Create heatmap and get the Axes object

ax = sns.heatmap(data)


# Add title

ax.set_title('Heatmap Title')


plt.show()
```
Output:



3. Using Matplotlib's `suptitle` Method

For more complex plots involving multiple subplots, you can use Matplotlib's `plt.suptitle()` method to add a title to the entire figure.

```
import matplotlib.pyplot as plt

import seaborn as sns


# Create figure and subplots
```

```
fig, ax = plt.subplots()



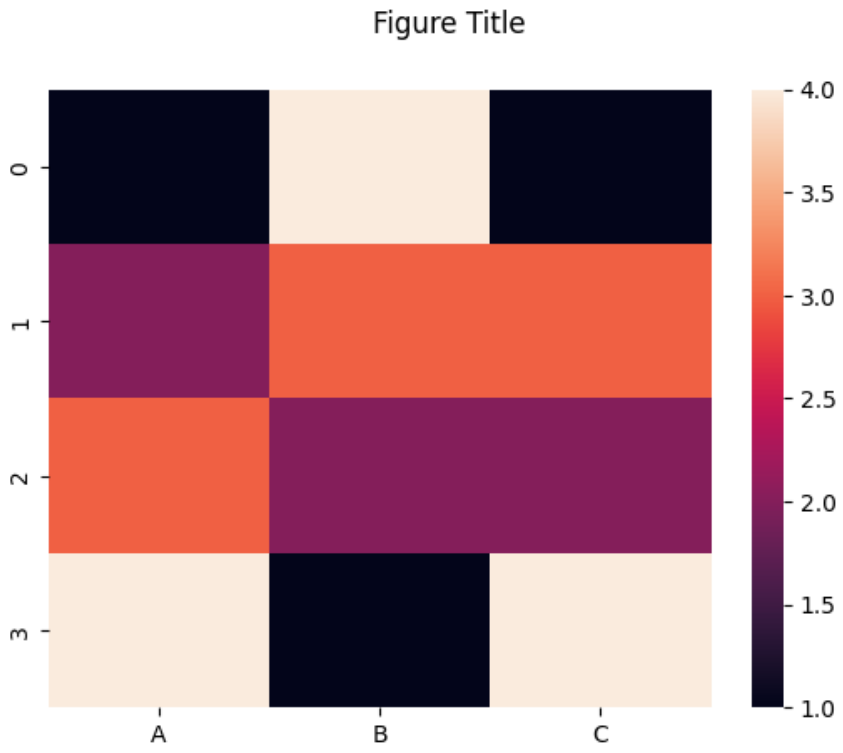
# Create heatmap

sns.heatmap(data, ax=ax)


# Add title to the figure

fig.suptitle('Figure Title')

plt.show()
```
Output:



Power BI: Business Analytics and Visualization,

Power BI is a **Data Visualization** and **Business Intelligence too**l by Microsoft that converts data from different data sources to create various **business intelligence reports**. Microsoft Power BI makes it easy for businesses to **spot trends**, **track performance**, and make **data-driven decisions**.

This **Power BI tutorial** is your one-stop guide for **learning Power BI from Scratch**. It covers all the basic and advanced concepts of Power BI like, filter Power BI, Power BI desktop, functions in Power BI, Power BI services, etc.

Whether you are a beginner or an experienced professional, this **free Power BI tutorial** will help you learn about all the Important concepts of Power BI with ease.

Power BI is a powerful business analytics tool developed by Microsoft that enables organizations to transform raw data into insightful and interactive visualizations, reports, and dashboards. It allows users to make data-driven decisions by providing a comprehensive, self-service platform for data analysis, reporting, and visualization. Here's a detailed breakdown of Power BI, focusing on its business analytics and visualization capabilities:

1. Business Analytics with Power BI

Business analytics in Power BI refers to the process of analyzing data to uncover valuable insights that help businesses make informed decisions. This process includes collecting, cleaning, transforming, and visualizing data from various sources.

Key components of business analytics in Power BI include:

a. Data Collection and Integration

Power BI allows users to connect to a wide variety of data sources, both internal and external, such as:

- **Databases:** SQL Server, Oracle, MySQL, etc.
- **Cloud Services:** Azure, Google Analytics, Salesforce, etc.
- **Spreadsheets:** Excel, CSV, etc.
- **APIs and Web Services:** Connecting to web data sources like APIs.
- **Big Data:** Azure Synapse Analytics, Hadoop, etc.

Power BI can combine data from these disparate sources into a single report or dashboard, providing a holistic view of the business.

b. Data Transformation and Cleaning (Power Query)

Power BI includes a built-in tool called **Power Query** that enables data transformation and cleaning. Users can:

- **Shape and transform** data (e.g., filter, sort, merge, group data).
- **Cleanse** data by handling null values, correcting errors, and eliminating duplicates.
- **Combine data from different sources** and create relationships between them.

This allows users to prepare data for further analysis and visualization without needing advanced programming skills.

c. Data Modeling (Power BI Desktop)

Once data is collected and cleaned, it can be modeled using Power BI Desktop. Data modeling involves creating relationships between different tables, defining key metrics (measures), and organizing the data into a coherent structure. This allows users to perform more complex analysis, such as:

- **Creating calculated columns and measures** using DAX (Data Analysis Expressions).
- **Building relationships** between tables to ensure the correct context when performing calculations.
- **Defining hierarchies** (e.g., year > quarter > month > day) for more granular analysis.

d. Analysis and Insights

Power BI supports various types of analysis, such as:

- **Trend Analysis**: Identifying patterns over time (e.g., sales growth).
- **Comparative Analysis**: Comparing performance across different categories or time periods.
- **Predictive Analytics**: Leveraging machine learning models to predict future trends (integrated with Azure Machine Learning or R scripts).
- **What-If Analysis**: Simulating different scenarios using Power BI's "What If" parameters and DAX formulas.

2. Visualization in Power BI

Visualization is one of Power BI's core strengths, allowing users to present data in a visually appealing and interactive manner. The tool provides a range of visualization options to help users explore and interpret data effectively.

a. Types of Visualizations

Power BI offers a wide variety of built-in charts and visuals:

- **Bar and Column Charts:** Represent data comparisons.
- **Line and Area Charts:** Display trends over time.
- **Pie and Donut Charts:** Show proportions and percentages.
- **Scatter and Bubble Charts:** Show relationships between two or more variables.
- **Maps:** Visualize geographic data (including shape maps, filled maps, and ArcGIS maps).
- **Tables and Matrixes:** Present detailed data in tabular form.
- **KPI (Key Performance Indicator) Visuals:** Display metrics such as progress towards goals.
- **Waterfall Charts:** Show incremental changes in a value.
- **Gauge and Card Visuals:** Provide quick insights into single metrics.

Power BI also allows users to customize visuals by changing the appearance, color schemes, and styles, ensuring that the visualizations align with the company's branding and messaging.

b. Interactive Visualizations

Power BI excels in creating interactive dashboards. Users can click on elements (e.g., bars, columns, data points) to filter and drill down into data, making it easy to explore insights from different angles. Some key features include:

- **Drill-through:** Enables users to right-click on a data point and drill down to a more detailed report.
- **Cross-filtering and cross-highlighting:** Clicking on one visual will filter or highlight relevant data in other visuals.

- **Slicers:** Visual filters that let users select specific data points (e.g., time period, region) to focus on.
- **Bookmarks:** Save and share specific views of a report with customized filters or selections.

c. Custom Visuals

In addition to the standard visualizations, Power BI allows users to import custom visuals created by other developers or the community. This is useful for specialized chart types that may not be available in the default library, such as:

- **Heatmaps**
- **Funnel charts**
- **Network diagrams**

Users can download these custom visuals from the Power BI marketplace or even create their own visuals using the Power BI developer tools.

3. Sharing and Collaboration

Once the data is analyzed and visualized, Power BI makes it easy to share insights with others within the organization. Power BI offers several options for sharing reports and dashboards:

- **Power BI Service (Cloud):** Publish reports to the Power BI service (cloud) for sharing and collaboration. Users can create workspaces, share reports, and set permissions.
- **Power BI Mobile App:** Access reports and dashboards on the go through Power BI's mobile app, available for iOS and Android.
- **Embedding:** Power BI allows embedding reports into web pages or third-party applications, making it easy to integrate analytics into business workflows.
- **Power BI for Teams:** Integration with Microsoft Teams allows users to view and interact with Power BI reports within the Teams environment, enhancing collaboration.

4. Security and Governance

Power BI provides robust security features to ensure that sensitive data is protected:

- **Row-level security (RLS):** Restricts data access for certain users based on their role.
- **Data encryption:** Ensures data is encrypted both in transit and at rest.
- **Auditing and monitoring:** Power BI tracks and logs user activity, which helps organizations monitor who accessed reports and data.
- **Compliance standards:** Power BI complies with a wide range of industry standards and regulations, including GDPR, ISO, and SOC.

5. Integration with Other Tools

Power BI integrates well with other Microsoft tools like:

- **Excel:** Import Excel files and even use Power Query and Power Pivot functionality.

- **Azure:** Direct integration with Azure services such as Azure Synapse Analytics and Azure Machine Learning.
- **Microsoft Teams:** As mentioned, sharing and collaborating on Power BI reports within Microsoft Teams.
- **PowerApps and Power Automate:** Power BI integrates with these tools to automate workflows and build custom business apps.

Features of Power BI

Some most important Power BI features are given below :

- **Data Visualization**: Power BI is used by organizations to create interactive reports and dashboards that visualize data in various ways, including charts, graphs, and maps.
- **Collaboration**: Power BI allows multiple users to work on the same report or dashboard at the same time. This makes it easy for teams to collaborate, share data and share insights.
- **Data Transformation:** Power BI can transform raw data into useful information that is a more usable format by cleaning and manipulating data. This allows businesses to work with clean, accurate data that is easy to analyze.
- **Integration**: Power BI integrates with a wide range of other Microsoft products or applications like Excel, SharePoint, and Teams. It helps in the workflow of the projects.
- **APIs for integration**. This feature provides developers with sample code and APIs for embedding the Power BI dashboard in other software products.

Power BI Architecture

The whole process of data sourcing to the creation of reports and dashboards consists of four basic steps. Many programs and processes work together to get the required results.

1. **Sourcing Data –** Power BI has a large range of data sources. Data can be imported from cloud-based online sources, and files in your system. There is a limit of 1 GB on importing data from online services. Power BI data sources are Excel, Text/CSV, XML, JSON, Oracle Database, Azure SQL Database, etc.
2. **Transforming the Data –** Cleaning and pre-processing are done before visualizing the data. It means removing missing values and useless data from the rows and columns. There are some rules to transforming and loading the datasets into the warehouse.
3. **Report and Publish –** Now, the report is created based on requirements, after cleaning and transforming the data. A report is data visualization with different filters and constraints represented in the form of graphs, pie, charts, etc.
4. **Creating Dashboards –** Power BI dashboards are created by pinning independent elements of live reports. It is created after the publishing of the report to the BI service. After reports get saved, the visual maintains the filter settings.

Power BI Architecture Components

- **Power Query:** Power Query is included in the Power Query Editor of Power BI Desktop, and It allows users to connect distinct information from multiple sources and convert them to satisfy their business requirements
- **Power BI service:** Power BI Service connects Power View, Power Pivot, Power BI Report Server, Power Q&A, and other components with the Workspace and allows you to connect with the data.
- **Power Pivot:** Power Pivot is a data modeling technique that uses the Data Analysis Expression (DAX) language to create data models.
- **Power View:** Power View creates graphs, maps, charts, and other visuals with drag-and-drop features. Power View can connect and filter different data sources to make a report on a single device.

- **Power Map:** Power Map allows you to base on the country, longitude, and latitude, Bing Maps shows the exact geospatial visuals of complex business information.
- **Power BI Desktop:** Power BI Desktop brings everything on a single platform, including Power Pivot, Power Query, Power View, etc. It is free software that you can install on a PC or laptop to create dashboards and reports.
- **Power BI App:** Power BI mobile applications are available for iOS, Android devices, and Windows. It does not matter whether your data is stored on the cloud or on-premises servers, you can access reports, run queries, share reports, and also get personalized notifications.
- **Power Q&A:** Power Q&A uses Natural Language Processing and integrates it with Cortana (Microsoft's virtual assistant) to get the answers to your query.

Applications of Power BI

Microsoft Power BI is a powerful business intelligence tool that allows users to analyze data and share insights in the form of graphical representations. Some common practical applications of Power BI are:

1. Data Visualization
2. Data Management
3. Data Analytics With Internal Software Systems
4. Business Reporting
5. Provide complex data within software and apps
6. Streamline Organizational Processes
7. Data Collaboration and Sharing
8. Enhance the marketing
9. Real-time look at the company's financial performance with financial Dashboards
10. Create Consistent Reporting Standards

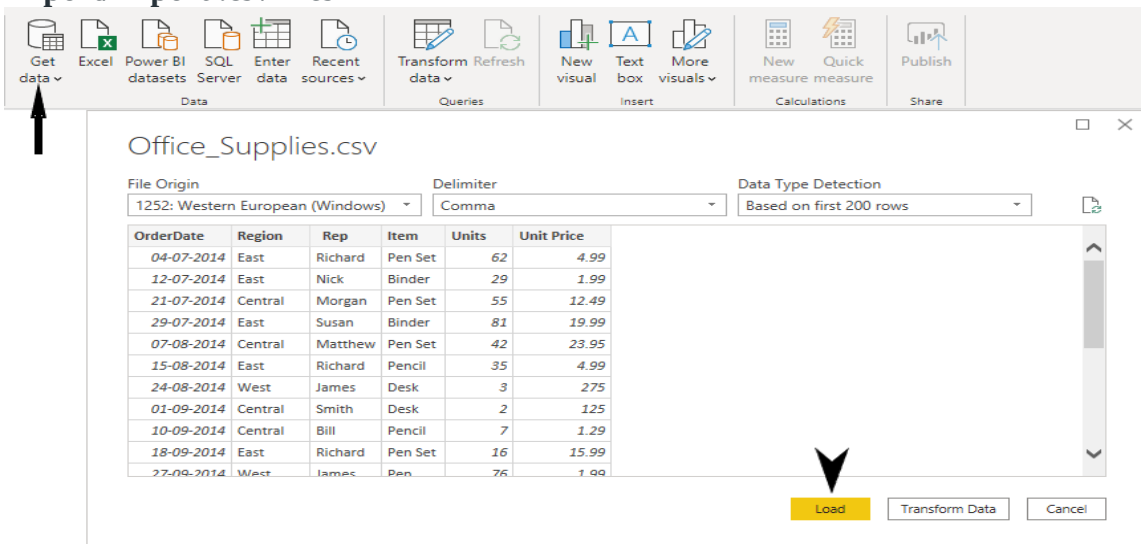
Power BI tools and functionality

Power BI is a Data Visualization and Business Intelligence tool by Microsoft that converts data from different data sources to create various business intelligence reports. It provides interactive visualizations using which end users can create reports and interactive dashboards by themselves. It is highly recommended to download Power-BI tool before proceeding further.

Navigating Power BI:

Power BI provides a plethora of various tool and services to make creative, interactive and intelligent Business Reports.

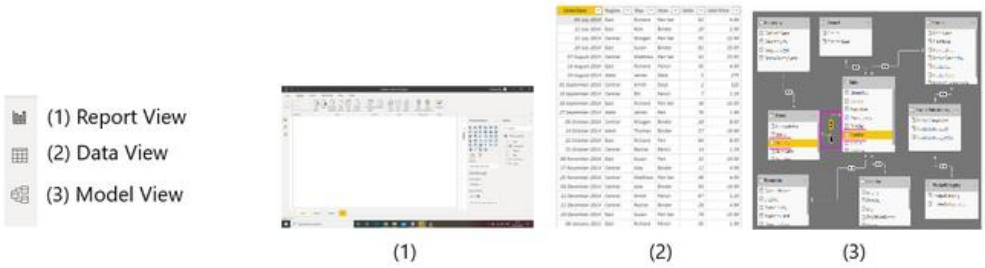
1. **Import/Export .csv files**



On following the above steps, the dataset will be uploaded and a window will pop up. This window is called the **Query Editor**.

What is Query Editor?
Query Editor in Power BI is used to **edit** or **format** the data files **before they are loaded** into the Power BI Model. The Query Editor plays the role of an intermediate data container where you can modify data type or the way the data is stored by selecting the particular rows and columns.

2. **Views in Power BI**
There are 3 different types of views in Power BI. You can switch between the 3 views using **Navigation Pane**.



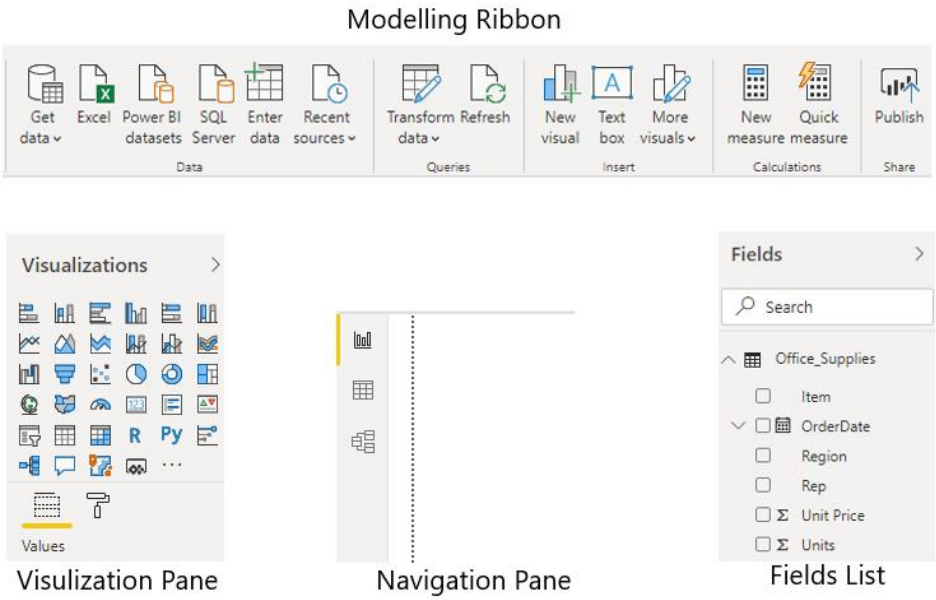
The 3 Views in Power BI

Report View:
- Report view is a section of Power-BI where you can create any number of report pages with visualizations.
- This view provides a designing environment where you can move visualizations around, copy and paste, merge, and so on.
- You can add one/multiple pages here for various visualization of BI-Reports.

Data View:
- Data view is a section of Power-BI that helps you inspect, explore, and understand data.
- It's different from how you view tables, columns, and data in Power Query Editor.
- With Data view, you're looking at your data **after it has been loaded** into the model.

Relationship View (Model View):
- Model view shows all of the tables, columns, and relationships in your model.
- This view can be especially helpful when your model has complex relationships between many tables.

3. **Various tools and panels in Power-BI**



The Power-BI provides various tools and services:

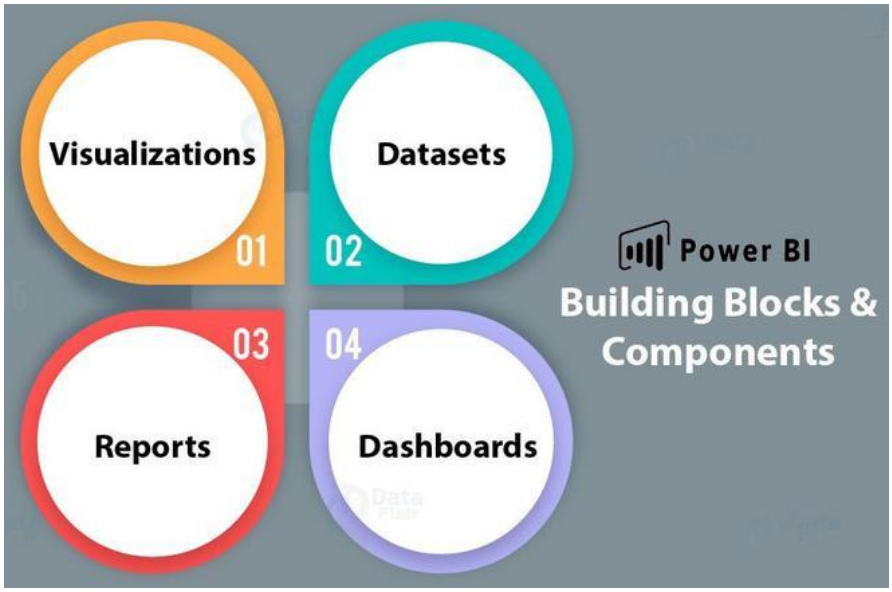
1. **Modelling Ribbon:**
   You can perform various functionalities using this panel.
   - Upload datasets.
   - Make calculations.
   - Edit data types.
   - Format data category for a column.
   - Insert visualizations.
2. **Fields list:** This list contains elements of your uploaded dataset. You can select a table or column to view in the data grid.
3. **Navigation Pane:** This panel controls the different views of Power BI (Report/Data/Model).
4. **Visualization Pane:** There are many different types of visualizations in Power-BI that help depict your report in various ways. This panel provides over 30 visualizations. More custom visualizations could be found online as well. Few important visualizations are listed below.

1	Stacked Bar/Column Charts	6	Tree Maps
2	Clustered Bar/Column Charts	7	Tables and Matrix
3	Line/Area Charts	8	Pie Charts
4	Ribbon Charts	9	Python-Scripts

5	Waterfall Charts	10	R-Scripts

Building Blocks of Power-BI

There are 4 major building blocks that make Power-BI a very powerful tool.



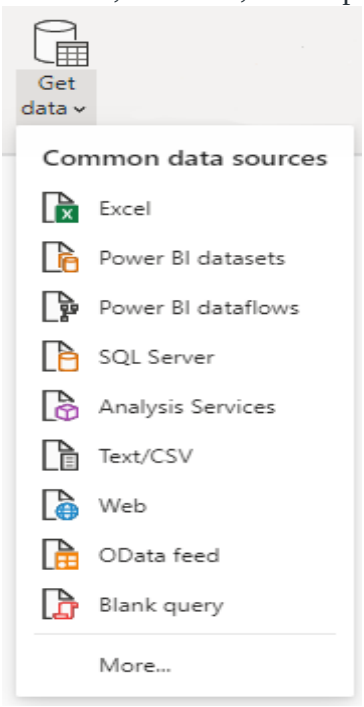
1. **Visualization**
   - A visualization is a visual representation of data, like a bar graph, pie chart, a color-coded map, or other through which you can visualize the data.
   - The following image shows a collection of different visualizations that are there in the Power BI.



Power BI : Visualizations

2. **Datasets:**
   - A Dataset is a collection of data or a container of data that is used by PowerBI to create visualizations.

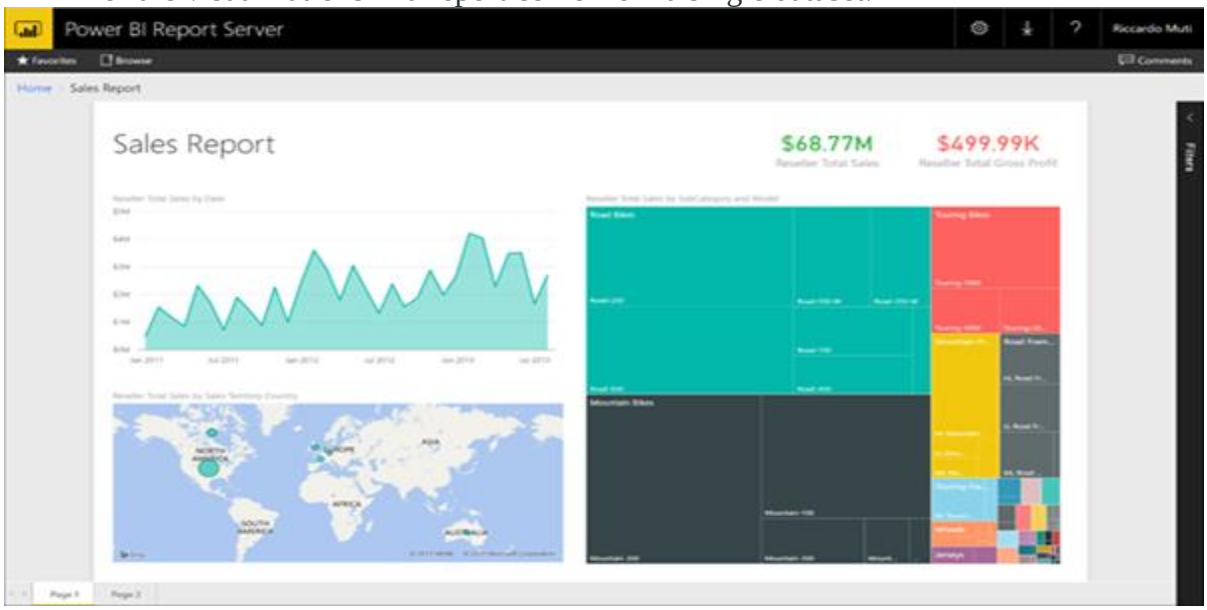
- A Dataset is something which you import or connect to.
- Datasets can be renamed, refreshed, removes, and explored.



Power BI : Types of Datasets

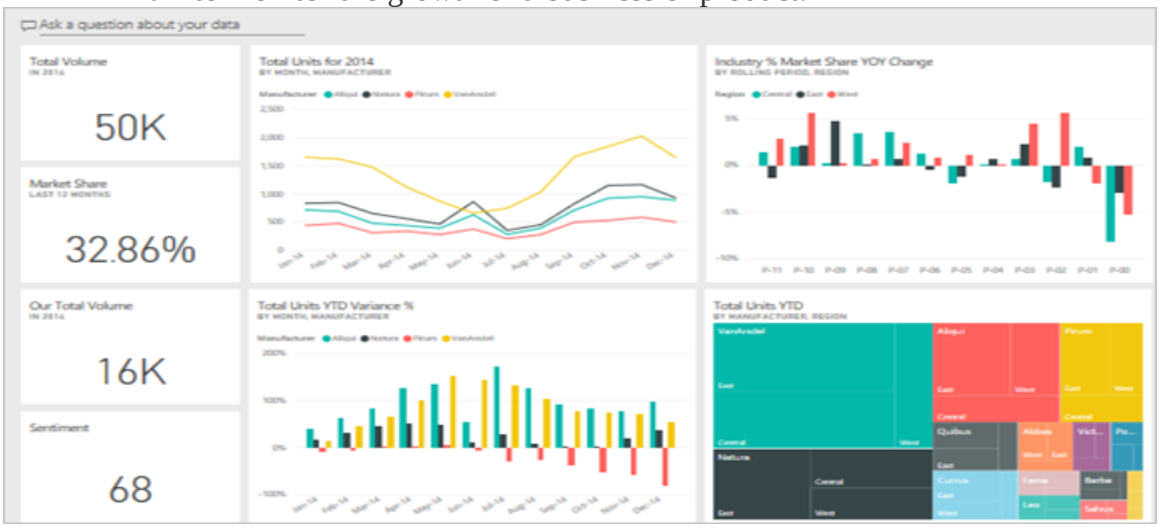
S.no	Data Source	Description
1.	Comma Separated Value (.csv)	A Comma Separated Values (.csv) file is a plain text file that contains a list of data. Every row can contain one or more values, which is separated by a comma.
2.	Excel (.xlsx)	A workbook can have data entered manually or data, which is queried and loaded from external data sources.
3.	Databases via SQL Server	You can connect directly to SQL Server Analysis Services Relational model databases.
4.	Databases on Cloud	You can connect live to Azure SQL Database, Azure SQL Data Warehouse, etc to get databases.
5.	Power BI (.pbi)	You can use Power BI Desktop to query and load data from external data sources.

3. **Reports:**
   - A report is one or more pages of interactive visuals, text, and graphics that makes up a single report.
   - All of the visualizations in a report come from a single dataset.



Power BI : Reports

4. **Dashboards:**
   - A dashboard represents a customized view of some subset of the underlying dataset(s).
   - A dashboard is a single canvas that has tiles, graphics, and text.
   - Purpose of Dashboards is to
     - get all the info at one place, needed to make decisions.
     - to monitor the most-important info about your business.
     - to ensure all colleagues are on the same page.
     - to monitor the growth of a business or product.



Power BI : Dashboards

This was a brief introduction covering the fundamentals of Power BI and the tools are services provided by Power BI. It is a very powerful tool by Microsoft that helps make large business decisions and stimulate the growth of the company. You can now import

your own dataset and create your own innovative and interactive reports using the building blocks of Power BI.

Linear regression and classification model .

What is Linear Regression?

Linear regression is a type of supervised machine learning algorithm that computes the linear relationship between the dependent variable and one or more independent features by fitting a linear equation to observed data.
When there is only one independent feature, it is known as Simple Linear Regression, and when there are more than one feature, it is known as Multiple Linear Regression.
Similarly, when there is only one dependent variable, it is considered Univariate Linear Regression, while when there are more than one dependent variables, it is known as Multivariate Regression.

Types of Linear Regression

There are two main types of linear regression:

Simple Linear Regression

This is the simplest form of linear regression, and it involves only one independent variable and one dependent variable. The equation for simple linear regression is:

$y=\beta0+\beta1X$ $y=\beta0+\beta1X$

where:

- Y is the dependent variable
- X is the independent variable
- β0 is the intercept
- β1 is the slope

Multiple Linear Regression

This involves more than one independent variable and one dependent variable. The equation for multiple linear regression is:

$y=\beta0+\beta1X1+\beta2X2+\ldots\ldots\beta nXn$ $y=\beta0+\beta1X1+\beta2X2+\ldots\ldots\beta nXn$

where:

- Y is the dependent variable
- X1, X2, …, Xn are the independent variables
- β0 is the intercept
- β1, β2, …, βn are the slopes

The goal of the algorithm is to find the best Fit Line equation that can predict the values based on the independent variables.

In regression set of records are present with X and Y values and these values are used to learn a function so if you want to predict Y from an unknown X this learned function can be used. In regression we have to find the value of Y, So, a function is required that predicts continuous Y in the case of regression given X as independent features.

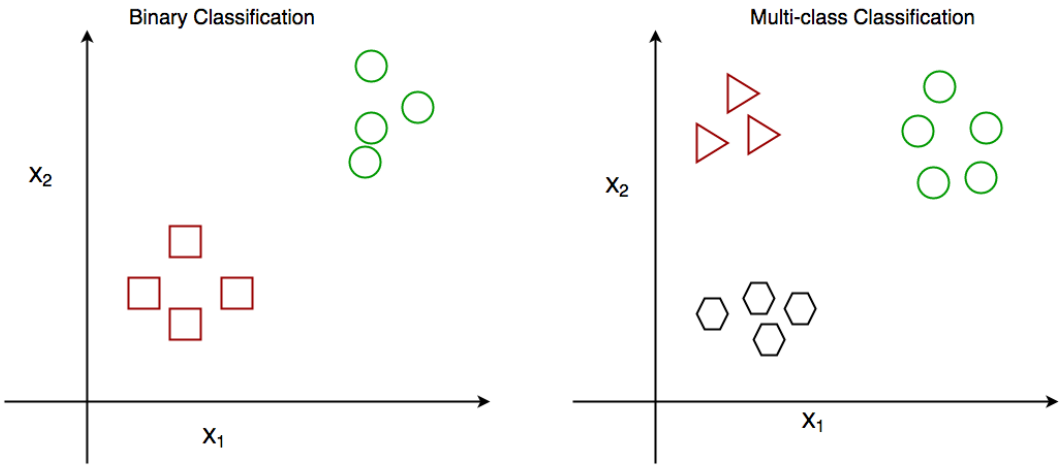
Classification:

Classification is the process of finding or discovering a model or function that helps in separating the data into multiple categorical classes i.e. discrete values. In classification, data is categorized under different labels according to some parameters given in the input and then the labels are predicted for the data.

- In a classification task, we are supposed to predict discrete target variables(class labels) using independent features.

- In the classification task, we are supposed to find a decision boundary that can separate the different classes in the target variable.

The derived mapping function could be demonstrated in the form of "IF-THEN" rules. The classification process deals with problems where the data can be divided into binary or multiple discrete labels. Let's take an example, suppose we want to predict the possibility of the winning of a match by Team A on the basis of some parameters recorded earlier. Then there would be two labels Yes and No.



Binary Classification and Multiclass Classification

Types of Classification Algorithms

There are different types of State of the art classification algorithms that have been developed over time to give the best results for classification tasks by employing techniques like bagging and boosting.

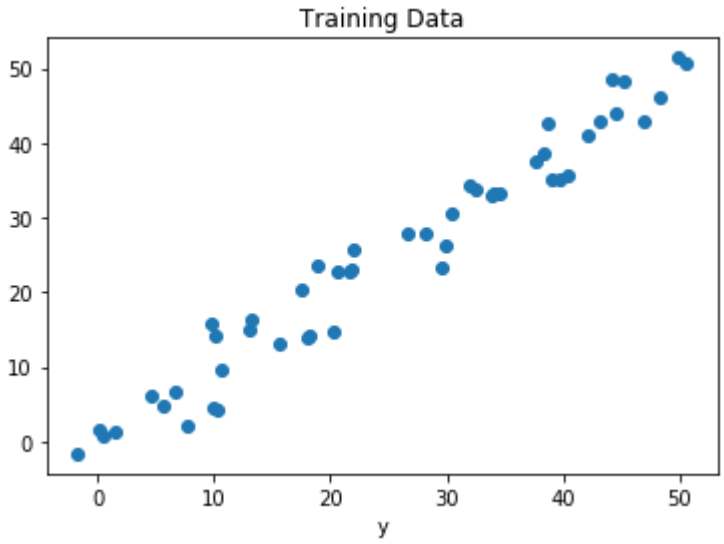
- Decision Tree
- Random Forest Classifier
- K – Nearest Neighbors
- Support Vector Machine

Regression Algorithms

Regression is the process of finding a model or function for distinguishing the data into continuous real values instead of using classes or discrete values. It can also identify the distribution movement depending on the historical data. Because a regression predictive model predicts a quantity, therefore, the skill of the model must be reported as an error in those predictions.

- In a regression task, we are supposed to predict a continuous target variable using independent features.
- In the regression tasks, we are faced with generally two types of problems linear and non-linear regression.

Let's take a similar example in regression also, where we are finding the possibility of rain in some particular regions with the help of some parameters recorded earlier. Then there is a probability associated with the rain.



Regression of Day vs Rainfall (in mm)

Types of Regression Algorithms

There are different types of State of the art regression algorithms that have been developed over time to give the best results for regression tasks by employing techniques like bagging and boosting.

- Lasso Regression
- Ridge Regression
- XGBoost Regressor
- LGBM Regressor

Comparison between Classification and Regression

Classification	Regression
In this problem statement, the target variables are discrete.	In this problem statement, the target variables are continuous.
Problems like Spam Email Classification, Disease prediction like problems are solved using Classification Algorithms.	Problems like House Price Prediction, Rainfall Prediction like problems are solved using regression Algorithms.
In this algorithm, we try to find the best possible decision boundary which can separate the two classes with the maximum possible separation.	In this algorithm, we try to find the best-fit line which can represent the overall trend in the data.
Evaluation metrics like Precision, Recall, and F1-Score are used here to evaluate the	Evaluation metrics like Mean Squared Error, R2-Score, and MAPE are used here

Classification	Regression
performance of the classification algorithms.	to evaluate the performance of the regression algorithms.
Here we face the problems like binary Classification or Multi-Class Classification problems.	Here we face the problems like Linear Regression models as well as non-linear models.
Input Data are Independent variables and categorical dependent variable.	Input Data are Independent variables and continuous dependent variable.
The classification algorithm's task mapping the input value of x with the discrete output variable of y.	The regression algorithm's task is mapping input value (x) with continuous output variable (y).
Output is Categorical labels.	Output is Continuous numerical values.
Objective is to Predict categorical/class labels.	Objective is to Predicting continuous numerical values.
Example use cases are Spam detection, image recognition, sentiment analysis	Example use cases are Stock price prediction, house price prediction, demand forecasting.
Examples of classification algorithms are: Logistic Regression, Decision Trees, Random Forest, Support Vector Machines (SVM), K-Nearest Neighbors (K-NN), Naive Bayes, Neural Networks, K-Means Clustering, Multi-layer Perceptron (MLP), etc.	**Examples of regression algorithms are:** Linear Regression, Polynomial Regression, Ridge Regression, Lasso Regression, Support Vector Regression (SVR), Decision Trees for Regression, Random Forest Regression, K-Nearest Neighbors (K-NN) Regression, Neural Networks for Regression, etc.